

The Concise TypeScript Book

Simone Poggiali

O Livro Conciso de TypeScript

O Livro Conciso de TypeScript fornece uma visão geral abrangente e concisa dos recursos do TypeScript. Ele oferece explicações claras que abrangem todos os aspectos encontrados na versão mais recente da linguagem, desde o seu poderoso sistema de tipos até recursos avançados. Seja você um iniciante ou um desenvolvedor experiente, este livro é um recurso inestimável para aprimorar sua compreensão e proficiência em TypeScript.

Este livro é completamente gratuito e de código aberto (open source).

Acredito que a educação técnica de alta qualidade deve ser acessível a todos, por isso mantenho este livro gratuito e aberto.

Se o livro ajudou você a resolver um bug, entender um conceito difícil ou avançar em sua carreira, considere apoiar meu trabalho pagando quanto quiser (preço sugerido: 15 USD) ou patrocinando um café. Seu apoio me ajuda a manter o conteúdo atualizado e a expandi-lo com novos exemplos e explicações mais profundas.



BUY ME A COFFEE

Donate 

Traduções

Este livro foi traduzido para vários idiomas, incluindo:

Chinês

Italiano

Português (Brasil)

Sueco

Downloads e site

Você também pode baixar a versão EPUB:

<https://github.com/gibbok/typescript-book/tree/main/downloads>

Uma versão online está disponível em:

<https://gibbok.github.io/typescript-book>

Sumário

- O Livro Conciso de TypeScript
 - Traduções
 - Downloads e site
 - Sumário
 - Introdução
 - Sobre o Autor
 - Introdução ao TypeScript
 - O que é TypeScript?
 - Por que TypeScript?
 - TypeScript e JavaScript

- Geração de Código TypeScript
- JavaScript Moderno Agora (Downleveling)
- Começando com TypeScript
 - Instalação
 - Configuração
 - Arquivo de Configuração do TypeScript
 - target
 - lib
 - strict
 - module
 - moduleResolution
 - esModuleInterop
 - jsx
 - skipLibCheck
 - files
 - include
 - exclude
 - importHelpers
 - Conselhos para Migração para TypeScript
- Explorando o Sistema de Tipos
 - O Serviço de Linguagem do TypeScript
 - Tipagem Estrutural
 - Regras Fundamentais de Comparação do TypeScript
 - Tipos como Conjuntos
 - Atribuir um tipo: Declarações de Tipo e Asserções de Tipo
 - Declaração de Tipo
 - Asserção de Tipo
 - Declarações de Ambiente (Ambient Declarations)
 - Verificação de Propriedades e Verificação de Excesso de Propriedades
 - Tipos Fracos (Weak Types)
 - Verificação Estrita de Objeto Literal (Freshness)
 - Inferência de Tipo
 - Inferências Mais Avançadas
 - Alargamento de Tipo (Type Widening)

- Const
 - Modificador Const em Parâmetros de Tipo
 - Asserção Const
- Anotação de Tipo Explícita
- Estreitamento de Tipo (Type Narrowing)
 - Condições
 - Lançando ou retornando
 - União Discriminada
 - Proteções de Tipo Definidas pelo Usuário (User-Defined Type Guards)
 - Redução de tipos com switch-true
- Tipos Primitivos
 - string
 - boolean
 - number
 - bigint
 - Symbol
 - null e undefined
 - Array
 - any
- Anotações de Tipo
- Propriedades Opcionais
- Propriedades Somente Leitura (Readonly)
- Assinaturas de Índice (Index Signatures)
- Estendendo Tipos
- Tipos Literais
- Inferência Literal
- strictNullChecks
- Enums
 - Enums numéricos
 - Enums de string
 - Enums constantes
 - Mapeamento reverso
 - Enums de ambiente
 - Membros computados e constantes
- Estreitamento (Narrowing)

- typeof type guards
- Estreitamento de veracidade (Truthiness narrowing)
- Estreitamento de igualdade (Equality narrowing)
- Estreitamento com operador In
- Estreitamento com instanceof
- Atribuições
- Análise de Fluxo de Controle
- Predicados de Tipo
- Unões Discriminadas
- O tipo never
- Verificação de exaustividade
- Tipos de Objeto
- Tipo Tupla (Anônimo)
- Tipo Tupla Nomeado (Rotulado)
- Tupla de Comprimento Fixo
- Tipo União
- Tipos de Interseção
- Indexação de Tipo
- Tipo a partir de Valor
- Tipo a partir de Retorno de Função
- Tipo a partir de Módulo
- Tipos Mapeados
- Modificadores de Tipos Mapeados
- Tipos Condicionais
- Tipos Condicionais Distributivos
- Inferência de tipo infer em Tipos Condicionais
- Tipos Condicionais Predefinidos
- Tipos de União de Template
- Tipo Any
- Tipo Unknown
- Tipo Void
- Tipo Never
- Interface e Tipo
 - Sintaxe Comum
 - Tipos Básicos
 - Objetos e Interfaces

- Tipos União e Interseção
- Primitivos de Tipo Integrados
- Objetos JS Integrados Comuns
- Sobrecargas
- Mesclagem e Extensão
- Diferenças entre Type e Interface
 - Classe
 - Sintaxe Comum de Classe
 - Construtor
 - Construtores Privados e Protegidos
 - Modificadores de Acesso
 - Get e Set
 - Auto-Accessors em Classes
 - this
 - Propriedades de Parâmetro
 - Classes Abstratas
 - Com Genéricos
 - Decoradores
 - Decoradores de Classe
 - Decorador de Propriedade
 - Decorador de Método
 - Decoradores de Getter e Setter
 - Metadados de Decorador
 - Herança
 - Estáticos
 - Inicialização de propriedade
 - Sobrecarga de método
- Genéricos
 - Tipo Genérico
 - Classes Genéricas
 - Restrições Genéricas
 - Estreitamento contextual genérico
- Tipos Estruturais Apagados (Erased Structural Types).
- Namespacing
- Símbolos
- Diretivas de Barra Tripla

- Manipulação de Tipos
 - Criando Tipos a partir de Tipos
 - Tipos de Acesso Indexado
 - Tipos Utilitários
 - Awaited<T>
 - Partial<T>
 - Required<T>
 - Readonly<T>
 - Record<K, T>
 - Pick<T, K>
 - Omit<T, K>
 - Exclude<T, U>
 - Extract<T, U>
 - NonNullable<T>
 - Parameters<T>
 - ConstructorParameters<T>
 - ReturnType<T>
 - InstanceType<T>
 - ThisParameterType<T>
 - OmitThisParameter<T>
 - ThisType<T>
 - Uppercase<T>
 - Lowercase<T>
 - Capitalize<T>
 - Uncapitalize<T>
 - NoInfer<T>
- Outros
 - Tratamento de Erros e Exceções
 - Classes Mixin
 - Recursos de Linguagem Assíncronos
 - Iteradores e Geradores
 - Referência JSDoc TsDocs
 - @types
 - JSX
 - Módulos ES6
 - Operador de Exponenciação ES7

- A instrução for-await-of
- Nova meta-propriedade target
- Expressões de Importação Dinâmica
- “tsc –watch”
- Operador de Asserção Não-nula
- Declarações padronizadas
- Encadeamento Opcional (Optional Chaining)
- Operador de coalescência nula
- Tipos de Literais de Template
- Sobrecarga de função
- Tipos Recursivos
- Tipos Condicionais Recursivos
- Suporte a Módulo ECMAScript no Node
- Funções de Asserção
- Tipos de Tupla Variádicos
- Tipos Boxed
- Covariância e Contravariância no TypeScript
 - Anotações de Variância Opcionais para Parâmetros de Tipo
- Assinaturas de Índice de Padrão de String de Template
- O Operador satisfies
- Importações e Exportações Apenas de Tipo
- Declaração using e Gerenciamento Explícito de Recursos
 - Declaração await using
- Atributos de Importação
- Verificação de Sintaxe de Expressões Regulares
- import defer

Introdução

Bem-vindo ao Livro Conciso de TypeScript! Este guia o equipa com conhecimentos essenciais e habilidades práticas para o desenvolvimento eficaz em TypeScript. Descubra conceitos e técnicas fundamentais para escrever código limpo e robusto. Seja você um

iniciante ou um desenvolvedor experiente, este livro serve tanto como um guia abrangente quanto como uma referência prática para aproveitar o poder do TypeScript em seus projetos.

Este livro cobre o TypeScript 6.0.

Sobre o Autor

Simone Poggiali é um Engenheiro Staff experiente com paixão por escrever código de nível profissional desde os anos 90. Ao longo de sua carreira internacional, contribuiu para inúmeros projetos para uma ampla gama de clientes, de startups a grandes organizações. Empresas notáveis como HelloFresh, Siemens, O2, Leroy Merlin e Snowplow se beneficiaram de sua expertise e dedicação.

Você pode encontrar Simone Poggiali nas seguintes plataformas:

- LinkedIn: <https://www.linkedin.com/in/simone-poggiali>
- GitHub: <https://github.com/gibbok>
- X.com: https://x.com/gibbok_coding
- Email: gibbok.coding@gmail.com

Lista completa de colaboradores:

<https://github.com/gibbok/typescript-book/graphs/contributors>

Introdução ao TypeScript

O que é TypeScript?

TypeScript é uma linguagem de programação fortemente tipada que se baseia no JavaScript. Foi originalmente projetada por Anders Hejlsberg em 2012 e é atualmente desenvolvida e mantida pela Microsoft como um projeto de código aberto.

O TypeScript compila para JavaScript e pode ser executado em qualquer ambiente de execução JavaScript (por exemplo, um navegador ou Node.js em um servidor).

Ele suporta múltiplos paradigmas de programação, como funcional, genérica, imperativa e orientada a objetos, e é uma linguagem compilada (transpilada) que é convertida em JavaScript antes da execução.

Por que TypeScript?

TypeScript é uma linguagem fortemente tipada que ajuda a prevenir erros comuns de programação e a evitar certos tipos de erros em tempo de execução antes que o programa seja executado.

Uma linguagem fortemente tipada permite ao desenvolvedor especificar várias restrições e comportamentos do programa nas definições de tipos de dados, facilitando a capacidade de verificar a correção do software e prevenir defeitos. Isso é especialmente valioso em aplicações de larga escala.

Alguns dos benefícios do TypeScript:

- Tipagem estática, opcionalmente fortemente tipada
- Inferência de Tipo
- Acesso a recursos ES6 e ES7
- Compatibilidade multiplataforma e entre navegadores
- Suporte de ferramentas com IntelliSense

TypeScript e JavaScript

Arquivos TypeScript são escritos em arquivos `.ts` ou `.tsx`, enquanto arquivos JavaScript são escritos em `.js` ou `.jsx`.

Arquivos com a extensão `.tsx` ou `.jsx` podem conter a Extensão de Sintaxe JavaScript JSX, que é usada no React para desenvolvimento de UI.

O TypeScript é um superconjunto tipado de JavaScript (ECMAScript 2015) em termos de sintaxe. Todo código JavaScript é código TypeScript válido, mas o inverso nem sempre é verdadeiro.

Por exemplo, considere uma função em um arquivo JavaScript com a extensão `.js`, como a seguinte:

```
const sum = (a, b) => a + b;
```

A função pode ser convertida e usada no TypeScript alterando a extensão do arquivo para `.ts`. No entanto, se a mesma função for anotada com tipos TypeScript, ela não poderá ser executada em nenhum ambiente de execução JavaScript sem compilação. O seguinte código TypeScript produzirá um erro de sintaxe se não for compilado:

```
const sum = (a: number, b: number): number => a + b;
```

O TypeScript foi projetado para detectar possíveis exceções que podem ocorrer em tempo de execução durante o tempo de compilação, fazendo com que o desenvolvedor defina a intenção com anotações de tipo. Além disso, o TypeScript também pode capturar problemas se nenhuma anotação de tipo for fornecida. Por exemplo, o seguinte trecho de código não especifica nenhum tipo TypeScript:

```
const items = [{ x: 1 }, { x: 2 }];  
const result = items.filter(item => item.y);
```

Neste caso, o TypeScript detecta um erro e informa:

```
Property 'y' does not exist on type '{ x: number; }'.
```

O sistema de tipos do TypeScript é amplamente influenciado pelo comportamento de tempo de execução do JavaScript. Por exemplo, o

operador de adição (+), que no JavaScript pode realizar a concatenação de strings ou a adição numérica, é modelado da mesma forma no TypeScript:

```
const result = '1' + 1; // Result is of type string
```

A equipe por trás do TypeScript tomou a decisão deliberada de sinalizar o uso incomum do JavaScript como erros. Por exemplo, considere o seguinte código JavaScript válido:

```
const result = 1 + true; // In JavaScript, the result is equal 2
```

No entanto, o TypeScript lança um erro:

```
Operator '+' cannot be applied to types 'number' and 'boolean'.
```

Este erro ocorre porque o TypeScript impõe estritamente a compatibilidade de tipos e, neste caso, identifica uma operação inválida entre um número e um booleano.

Geração de Código TypeScript

O compilador TypeScript tem duas responsabilidades principais: verificar se há erros de tipo e compilar para JavaScript. Esses dois processos são independentes um do outro. Os tipos não afetam a execução do código em um ambiente de execução JavaScript, pois são completamente apagados durante a compilação. O TypeScript ainda pode gerar JavaScript mesmo na presença de erros de tipo. Aqui está um exemplo de código TypeScript com um erro de tipo:

```
const add = (a: number, b: number): number => a + b;  
const result = add('x', 'y'); // Argument of type 'string' is not  
assignable to parameter of type 'number'.
```

No entanto, ele ainda pode produzir uma saída JavaScript executável:

```
'use strict';
const add = (a, b) => a + b;
const result = add('x', 'y'); // xy
```

Não é possível verificar tipos TypeScript em tempo de execução. Por exemplo:

```
interface Animal {
  name: string;
}
interface Dog extends Animal {
  bark: () => void;
}
interface Cat extends Animal {
  meow: () => void;
}
const makeNoise = (animal: Animal) => {
  if (animal instanceof Dog) {
    // 'Dog' only refers to a type, but is being used as a
    value here.
    // ...
  }
};
```

Como os tipos são apagados após a compilação, não há como executar este código em JavaScript. Para reconhecer tipos em tempo de execução, precisamos usar outro mecanismo. O TypeScript fornece várias opções, sendo uma comum a “união tagueada” (tagged union). Por exemplo:

```
interface Dog {
  kind: 'dog'; // Tagged union
  bark: () => void;
}
interface Cat {
  kind: 'cat'; // Tagged union
  meow: () => void;
}
type Animal = Dog | Cat;

const makeNoise = (animal: Animal) => {
  if (animal.kind === 'dog') {
```

```

        animal.bark();
    } else {
        animal.meow();
    }
};

const dog: Dog = {
    kind: 'dog',
    bark: () => console.log('bark'),
};
makeNoise(dog);

```

A propriedade “kind” é um valor que pode ser usado em tempo de execução para distinguir entre objetos em JavaScript.

Também é possível que um valor em tempo de execução tenha um tipo diferente daquele declarado na declaração de tipo. Por exemplo, se o desenvolvedor interpretou mal um tipo de API e o anotou incorretamente.

O TypeScript é um superconjunto do JavaScript, portanto a palavra-chave “class” pode ser usada como um tipo e valor em tempo de execução.

```

class Animal {
    constructor(public name: string) {}
}
class Dog extends Animal {
    constructor(
        public name: string,
        public bark: () => void
    ) {
        super(name);
    }
}
class Cat extends Animal {
    constructor(
        public name: string,
        public meow: () => void
    ) {
        super(name);
    }
}

```

```

}
type Mammal = Dog | Cat;

const makeNoise = (mammal: Mammal) => {
  if (mammal instanceof Dog) {
    mammal.bark();
  } else {
    mammal.meow();
  }
};

const dog = new Dog('Fido', () => console.log('bark'));
makeNoise(dog);

```

No JavaScript, uma “classe” tem uma propriedade “prototype”, e o operador “instanceof” pode ser usado para testar se a propriedade prototype de um construtor aparece em qualquer lugar na cadeia de protótipos de um objeto.

O TypeScript não tem efeito no desempenho em tempo de execução, pois todos os tipos serão apagados. No entanto, o TypeScript introduz alguma sobrecarga no tempo de compilação.

JavaScript Moderno Agora (Downleveling)

O TypeScript pode compilar código para qualquer versão lançada do JavaScript desde o ECMAScript 3 (1999). Isso significa que o TypeScript pode transpilar o código dos recursos JavaScript mais recentes para versões mais antigas, um processo conhecido como *Downleveling*. Isso permite o uso do JavaScript moderno, mantendo a compatibilidade máxima com ambientes de execução mais antigos.

É importante notar que durante a transpilação para uma versão mais antiga do JavaScript, o TypeScript pode gerar código que pode incorrer em uma sobrecarga de desempenho em comparação com as implementações nativas.

Aqui estão alguns dos recursos modernos do JavaScript que podem ser usados no TypeScript:

- Módulos ECMAScript em vez de callbacks “define” no estilo AMD ou instruções “require” do CommonJS.
- Classes em vez de protótipos.
- Declaração de variáveis usando “let” ou “const” em vez de “var”.
- Loop “for-of” ou “.forEach” em vez do loop “for” tradicional.
- Funções de seta (Arrow functions) em vez de expressões de função.
- Atribuição via desestruturação (Destructuring assignment).
- Nomes de propriedade/método abreviados e nomes de propriedade computados.
- Parâmetros de função padrão.

Ao aproveitar esses recursos modernos do JavaScript, os desenvolvedores podem escrever códigos mais expressivos e concisos no TypeScript.

Começando com TypeScript

Instalação

O Visual Studio Code oferece excelente suporte para a linguagem TypeScript, mas não inclui o compilador TypeScript. Para instalar o compilador TypeScript, você pode usar um gerenciador de pacotes como npm ou yarn:

```
npm install typescript --save-dev
```

ou

```
yarn add typescript --dev
```

Certifique-se de realizar o commit do arquivo de bloqueio (lockfile) gerado para garantir que cada membro da equipe use a mesma versão do TypeScript.

Para executar o compilador TypeScript, você pode usar os seguintes comandos:

```
npx tsc
```

ou

```
yarn tsc
```

Recomenda-se instalar o TypeScript por projeto em vez de globalmente, pois fornece um processo de construção mais previsível. No entanto, para ocasiões pontuais, você pode usar o seguinte comando:

```
npx tsc
```

ou instalá-lo globalmente:

```
npm install -g typescript
```

Se você estiver usando o Microsoft Visual Studio, pode obter o TypeScript como um pacote no NuGet para seus projetos MSBuild. No Console do Gerenciador de Pacotes NuGet, execute o seguinte comando:

```
Install-Package Microsoft.TypeScript.MSBuild
```

Durante a instalação do TypeScript, dois executáveis são instalados: “tsc” como o compilador TypeScript e “tsserver” como o servidor autônomo do TypeScript. O servidor autônomo contém o compilador e os serviços de linguagem que podem ser utilizados por editores e IDEs para fornecer preenchimento automático de código.

Além disso, existem vários transpiladores compatíveis com TypeScript disponíveis, como Babel (via um plugin) ou swc. Esses

transpiladores podem ser usados para converter código TypeScript em outras linguagens ou versões de destino.

Configuração

O TypeScript pode ser configurado usando as opções da CLI do tsc ou utilizando um arquivo de configuração dedicado chamado tsconfig.json localizado na raiz do projeto.

Para gerar um arquivo tsconfig.json pré-preenchido com as configurações recomendadas, você pode usar o seguinte comando:

```
tsc --init
```

Ao executar o comando tsc localmente, o TypeScript compilará o código usando a configuração especificada no arquivo tsconfig.json mais próximo.

Aqui estão alguns exemplos de comandos da CLI que rodam com as configurações padrão:

```
tsc main.ts // Compila um arquivo específico (main.ts) para
JavaScript
tsc src/*.ts // Compila todos os arquivos .ts na pasta 'src' para
JavaScript
tsc app.ts util.ts --outfile index.js // Compila dois arquivos
TypeScript (app.ts e util.ts) em um único arquivo JavaScript
(index.js)
```

Arquivo de Configuração do TypeScript

Um arquivo tsconfig.json é usado para configurar o Compilador TypeScript (tsc). Geralmente, ele é adicionado à raiz do projeto, junto com o arquivo package.json.

Notas:

- tsconfig.json aceita comentários, mesmo estando no formato json.
- É aconselhável usar este arquivo de configuração em vez das opções de linha de comando.

No link a seguir você encontra a documentação completa e seu esquema:

<https://www.typescriptlang.org/tsconfig>

<https://www.typescriptlang.org/tsconfig/>

A seguir, apresentamos uma lista das configurações comuns e úteis:

target

A propriedade “target” é usada para especificar qual versão do JavaScript ECMAScript seu TypeScript deve emitir/compilar. Para navegadores modernos, o ES6 é uma boa opção; para navegadores mais antigos, o ES5 é recomendado. Observação: o suporte a ES5 foi removido no TypeScript 6.0.

lib

A propriedade “lib” é usada para especificar quais arquivos de biblioteca incluir no tempo de compilação. O TypeScript inclui automaticamente APIs para recursos especificados na propriedade “target”, mas é possível omitir ou escolher bibliotecas específicas para necessidades particulares. Por exemplo, se você estiver trabalhando em um projeto de servidor, pode excluir a biblioteca “DOM”, que é útil apenas em um ambiente de navegador.

strict

A opção “strict” aprimora a segurança de tipos, permitindo verificações mais rigorosas. Ela está habilitada por padrão a partir do

TypeScript 6.0; caso contrário, você deve defini-la explicitamente como `true` no seu arquivo `tsconfig.json`. Habilitar `strict` permite que o TypeScript:

- Emitir código usando `use strict` para cada arquivo de origem.
- Considerar `null` e `undefined` no processo de verificação de tipos.
- Desabilitar o uso do tipo `any` quando não houver anotações de tipo.
- Levantar um erro sobre o uso da expressão `this`, que de outra forma implicaria o tipo `any`.

module

A propriedade `module` define o sistema de módulos suportado pelo programa compilado. Durante a execução, um carregador de módulos é usado para localizar e executar as dependências com base no sistema de módulos especificado.

Os carregadores de módulos mais comuns usados em JavaScript são o CommonJS do Node.js para aplicações do lado do servidor e o RequireJS para módulos AMD em aplicações web baseadas em navegador. O TypeScript pode gerar código para vários sistemas de módulos, incluindo UMD, SystemJS, ESNext, ES2015/ES6 e ES2020. O sistema de módulos deve ser escolhido com base no ambiente de destino e no mecanismo de carregamento de módulos disponível nesse ambiente.

Nota: O suporte para sistemas de módulos mais antigos (AMD, UMD, SystemJS) foi removido no TypeScript 6.0.

moduleResolution

A propriedade `moduleResolution` especifica a estratégia de resolução de módulos. Use `node` para código TypeScript moderno;

a estratégia “classic” é usada apenas para versões antigas do TypeScript (antes da 1.6).

esModuleInterop

A propriedade “esModuleInterop” permite a importação padrão de módulos CommonJS que não exportaram usando a propriedade “default”; esta propriedade fornece um shim para garantir a compatibilidade no JavaScript emitido. Após habilitar esta opção, podemos usar `import MyLibrary from "my-library"` em vez de `import * as MyLibrary from "my-library"`.

Originalmente, a opção “esModuleInterop” era opcional para evitar alterações que quebrassem a compatibilidade, mas há muito tempo é o padrão recomendado. Desativá-la pode causar problemas sutis em tempo de execução ao usar CommonJS com ESM. Observação: a partir do TypeScript 6.0, esse comportamento de interoperabilidade mais seguro está sempre ativado.

jsx

A propriedade “jsx” aplica-se apenas a arquivos .tsx usados no ReactJS e controla como as construções JSX são compiladas em JavaScript. Uma opção comum é “preserve”, que compilará para um arquivo .jsx mantendo o JSX inalterado para que ele possa ser passado para diferentes ferramentas, como o Babel, para transformações posteriores.

skipLibCheck

A propriedade “skipLibCheck” evitará que o TypeScript verifique os tipos de todos os pacotes de terceiros importados. Esta propriedade reduzirá o tempo de compilação de um projeto. O TypeScript ainda verificará seu código em relação às definições de tipo fornecidas por esses pacotes.

files

A propriedade “files” indica ao compilador uma lista de arquivos que devem sempre ser incluídos no programa.

include

A propriedade “include” indica ao compilador uma lista de arquivos que gostaríamos de incluir. Esta propriedade permite padrões semelhantes a glob, como “*” *para qualquer subdiretório*, “*” para qualquer nome de arquivo e “?” para caracteres opcionais.

exclude

A propriedade “exclude” indica ao compilador uma lista de arquivos que não devem ser incluídos na compilação. Isso pode incluir arquivos como “node_modules” ou arquivos de teste. Nota: tsconfig.json permite comentários.

importHelpers

O TypeScript usa código auxiliar ao gerar código para certos recursos avançados do JavaScript durante o downleveling. Por padrão, esses auxiliares são duplicados nos arquivos que os utilizam. A opção importHelpers importa esses auxiliares do módulo tslib, tornando a saída do JavaScript mais eficiente.

Conselhos para Migração para TypeScript

Para projetos grandes, recomenda-se adotar uma transição gradual onde o código TypeScript e JavaScript coexistirão inicialmente. Apenas projetos pequenos podem ser migrados para TypeScript de uma só vez.

O primeiro passo desta transição é introduzir o TypeScript no processo da cadeia de construção. Isso pode ser feito usando a opção de compilador “allowJs”, que permite que arquivos .ts e .tsx coexistam com arquivos JavaScript existentes. Como o TypeScript voltará para um tipo “any” para uma variável quando não puder inferir o tipo dos arquivos JavaScript, recomenda-se desabilitar “noImplicitAny” em suas opções de compilador no início da migração.

O segundo passo é garantir que seus testes JavaScript funcionem junto com os arquivos TypeScript, para que você possa executar testes conforme converte cada módulo. Se estiver usando Jest, considere usar o `ts-jest`, que permite testar projetos TypeScript com Jest.

O terceiro passo é incluir declarações de tipo para bibliotecas de terceiros em seu projeto. Essas declarações podem ser encontradas empacotadas ou no DefinitelyTyped. Você pode pesquisar por elas usando <https://www.typescriptlang.org/dt/search> e instalá-las usando:

```
npm install --save-dev @types/package-name
```

ou

```
yarn add --dev @types/package-name
```

O quarto passo é migrar módulo por módulo com uma abordagem de baixo para cima, seguindo seu Gráfico de Dependências começando pelas folhas. A ideia é começar convertendo Módulos que não dependem de outros Módulos. Para visualizar os gráficos de dependência, você pode usar a ferramenta “madge”.

Bons módulos candidatos para essas conversões iniciais são funções utilitárias e código relacionado a APIs ou especificações externas. É possível gerar automaticamente definições de tipo TypeScript a

partir de contratos Swagger, GraphQL ou esquemas JSON para serem incluídos em seu projeto.

Quando não houver especificações ou esquemas oficiais disponíveis, você pode gerar tipos a partir de dados brutos, como JSON retornado por um servidor. No entanto, recomenda-se gerar tipos a partir de especificações em vez de dados para evitar perder casos extremos.

Durante a migração, evite a refatoração de código e concentre-se apenas em adicionar tipos aos seus módulos.

O quinto passo é habilitar o “noImplicitAny”, que forçará que todos os tipos sejam conhecidos e definidos, proporcionando uma melhor experiência de TypeScript para seu projeto.

Durante a migração, você pode usar a diretiva `@ts-check`, que habilita a verificação de tipos do TypeScript em um arquivo JavaScript. Esta diretiva fornece uma versão flexível de verificação de tipos e pode ser usada inicialmente para identificar problemas em arquivos JavaScript. Quando o `@ts-check` é incluído em um arquivo, o TypeScript tentará deduzir definições usando comentários no estilo JSDoc. No entanto, considere usar anotações JSDoc apenas em um estágio muito inicial da migração.

Considere manter o valor padrão de `noEmitOnError` no seu `tsconfig.json` como `false`. Isso permitirá gerar o código-fonte JavaScript mesmo se erros forem relatados.

Explorando o Sistema de Tipos

O Serviço de Linguagem do TypeScript

O Serviço de Linguagem do TypeScript, também conhecido como tsserver, oferece vários recursos, como relatório de erros, diagnósticos, compilar ao salvar, renomeação, ir para definição, listas de preenchimento, ajuda de assinatura e muito mais. É usado principalmente por ambientes de desenvolvimento integrados (IDEs) para fornecer suporte ao IntelliSense. Ele se integra perfeitamente ao Visual Studio Code e é utilizado por ferramentas como Conquer of Completion (Coc).

Os desenvolvedores podem aproveitar uma API dedicada e criar seus próprios plugins de serviço de linguagem personalizados para aprimorar a experiência de edição do TypeScript. Isso pode ser particularmente útil para implementar recursos especiais de linting ou habilitar o preenchimento automático para uma linguagem de modelagem personalizada.

Um exemplo de plugin personalizado do mundo real é o “typescript-styled-plugin”, que fornece relatórios de erros de sintaxe e suporte IntelliSense para propriedades CSS em componentes estilizados (styled components).

Para mais informações e guias de início rápido, você pode consultar o Wiki oficial do TypeScript no GitHub:

<https://github.com/microsoft/TypeScript/wiki/>

Tipagem Estrutural

O TypeScript é baseado em um sistema de tipos estrutural. Isso significa que a compatibilidade e a equivalência de tipos são determinadas pela estrutura ou definição real do tipo, em vez de seu

nome ou local de declaração, como em sistemas de tipos nominativos como C# ou C++.

O sistema de tipos estrutural do TypeScript foi projetado com base em como o sistema de tipagem dinâmica “duck typing” do JavaScript funciona durante o tempo de execução.

O exemplo a seguir é um código TypeScript válido. Como você pode observar, “X” e “Y” têm o mesmo membro “a”, embora tenham nomes de declaração diferentes. Os tipos são determinados por suas estruturas e, neste caso, como as estruturas são as mesmas, eles são compatíveis e válidos.

```
type X = {  
  a: string;  
};  
type Y = {  
  a: string;  
};  
const x: X = { a: 'a' };  
const y: Y = x; // Válido
```

Regras Fundamentais de Comparação do TypeScript

O processo de comparação do TypeScript é recursivo e executado em tipos aninhados em qualquer nível.

Um tipo “X” é compatível com “Y” se “Y” tiver pelo menos os mesmos membros que “X”.

```
type X = {  
  a: string;  
};  
const y = { a: 'A', b: 'B' }; // Válido, pois tem pelo menos os  
mesmos membros que X  
const r: X = y;
```

Os parâmetros da função são comparados por tipos, não por seus nomes:

```
type X = (a: number) => void;
type Y = (a: number) => void;
let x: X = (j: number) => undefined;
let y: Y = (k: number) => undefined;
y = x; // Válido
x = y; // Válido
```

Os tipos de retorno da função devem ser os mesmos:

```
type X = (a: number) => undefined;
type Y = (a: number) => number;
let x: X = (a: number) => undefined;
let y: Y = (a: number) => 1;
y = x; // Inválido
x = y; // Inválido
```

O tipo de retorno de uma função de origem deve ser um subtipo do tipo de retorno de uma função de destino:

```
let x = () => ({ a: 'A' });
let y = () => ({ a: 'A', b: 'B' });
x = y; // Válido
y = x; // Inválido, o membro b está faltando
```

Descartar parâmetros de função é permitido, pois é uma prática comum no JavaScript, por exemplo, usando “Array.prototype.map()”:

```
[1, 2, 3].map((element, _index, _array) => element + 'x');
```

Portanto, as seguintes declarações de tipo são completamente válidas:

```
type X = (a: number) => undefined;
type Y = (a: number, b: number) => undefined;
let x: X = (a: number) => undefined;
let y: Y = (a: number) => undefined; // Falta o parâmetro b
y = x; // Válido
```

Quaisquer parâmetros opcionais adicionais do tipo de origem são válidos:

```
type X = (a: number, b?: number, c?: number) => undefined;
type Y = (a: number) => undefined;
let x: X = a => undefined;
let y: Y = a => undefined;
y = x; // Válido
x = y; // Válido
```

Quaisquer parâmetros opcionais do tipo de destino sem parâmetros correspondentes no tipo de origem são válidos e não constituem um erro:

```
type X = (a: number) => undefined;
type Y = (a: number, b?: number) => undefined;
let x: X = a => undefined;
let y: Y = a => undefined;
y = x; // Válido
x = y; // Válido
```

O parâmetro rest é tratado como uma série infinita de parâmetros opcionais:

```
type X = (a: number, ...rest: number[]) => undefined;
let x: X = a => undefined; // Válido
```

Funções com sobrecargas são válidas se a assinatura da sobrecarga for compatível com sua assinatura de implementação:

```
function x(a: string): void;
function x(a: string, b: number): void;
function x(a: string, b?: number): void {
    console.log(a, b);
}
x('a'); // Válido
x('a', 1); // Válido
```

```
function y(a: string): void; // Inválido, não compatível com a
    assinatura de implementação
function y(a: string, b: number): void;
function y(a: string, b: number): void {
```

```

        console.log(a, b);
    }
    y('a');
    y('a', 1);

```

A comparação de parâmetros de função é bem-sucedida se os parâmetros de origem e de destino forem atribuíveis a supertipos ou subtipos (bivariância).

```

// Supertipo
class X {
    a: string;
    constructor(value: string) {
        this.a = value;
    }
}
// Subtipo
class Y extends X {}
// Subtipo
class Z extends X {}

type GetA = (x: X) => string;
const getA: GetA = x => x.a;

// A bivariância aceita supertipos
console.log(getA(new X('x'))); // Válido
console.log(getA(new Y('Y'))); // Válido
console.log(getA(new Z('z'))); // Válido

```

Enums são comparáveis e válidos com números e vice-versa, mas comparar valores de Enum de diferentes tipos de Enum é inválido.

```

enum X {
    A,
    B,
}
enum Y {
    A,
    B,
    C,
}
const xa: number = X.A; // Válido

```

```
const ya: Y = 0; // Válido
X.A === Y.A; // Inválido
```

Instâncias de uma classe estão sujeitas a uma verificação de compatibilidade para seus membros privados e protegidos:

```
class X {
  public a: string;
  constructor(value: string) {
    this.a = value;
  }
}
```

```
class Y {
  private a: string;
  constructor(value: string) {
    this.a = value;
  }
}
```

```
let x: X = new Y('y'); // Inválido
```

A verificação de comparação não leva em consideração as diferentes hierarquias de herança, por exemplo:

```
class X {
  public a: string;
  constructor(value: string) {
    this.a = value;
  }
}
class Y extends X {
  public a: string;
  constructor(value: string) {
    super(value);
    this.a = value;
  }
}
class Z {
  public a: string;
  constructor(value: string) {
    this.a = value;
  }
}
```

```

}
let x: X = new X('x');
let y: Y = new Y('y');
let z: Z = new Z('z');
x === y; // Válido
x === z; // Válido mesmo que z seja de uma hierarquia de herança
diferente

```

Genéricos são comparados usando suas estruturas baseadas no tipo resultante após a aplicação do parâmetro genérico; apenas o resultado final é comparado como um tipo não genérico.

```

interface X<T> {
  a: T;
}
let x: X<number> = { a: 1 };
let y: X<string> = { a: 'a' };
x === y; // Inválido, pois o argumento de tipo é usado na estrutura
final

```

```

interface X<T> {}
const x: X<number> = 1;
const y: X<string> = 'a';
x === y; // Válido, pois o argumento de tipo não é usado na
estrutura final

```

Quando os genéricos não têm seu argumento de tipo especificado, todos os argumentos não especificados são tratados como tipos com “any”:

```

type X = <T>(x: T) => T;
type Y = <K>(y: K) => K;
let x: X = x => x;
let y: Y = y => y;
x = y; // Válido

```

Lembre-se:

```

let a: number = 1;
let b: number = 2;
a = b; // Válido, tudo é atribuível a si mesmo

```

```

let c: any;
c = 1; // Válido, todos os tipos são atribuíveis a any

let d: unknown;
d = 1; // Válido, todos os tipos são atribuíveis a unknown

let e: unknown;
let e1: unknown = e; // Válido, unknown só é atribuível a si mesmo
e a any
let e2: any = e; // Válido
let e3: number = e; // Inválido

let f: never;
f = 1; // Inválido, nada é atribuível a never

let g: void;
let g1: any;
g = 1; // Inválido, void não é atribuível a nada, exceto any, nem
nada é atribuível a ele
g = g1; // Válido

```

Observe que quando “strictNullChecks” está habilitado, “null” e “undefined” são tratados de forma semelhante a “void”; caso contrário, são semelhantes a “never”.

Tipos como Conjuntos

No TypeScript, um tipo é um conjunto de valores possíveis. Este conjunto também é conhecido como o domínio do tipo. Cada valor de um tipo pode ser visto como um elemento em um conjunto. Um tipo estabelece as restrições que cada elemento no conjunto deve satisfazer para ser considerado um membro desse conjunto. A principal tarefa do TypeScript é verificar se um conjunto é um subconjunto de outro.

O TypeScript suporta vários tipos de conjuntos:

Termo do conjunto	TypeScript	Notas
Conjunto vazio	never	“never” não contém nada além de si mesmo
Conjunto de elemento único	undefined / null / tipo literal	
Conjunto finito	boolean / união	
Conjunto infinito	string / number / objeto	
Conjunto universal	any / unknown	Cada elemento é um membro de “any” e cada conjunto é um subconjunto dele / “unknown” é uma contraparte segura em termos de tipo do “any”

Aqui estão alguns exemplos:

TypeScript	Termo do conjunto	Exemplo
never	∅ (conjunto vazio)	const x: never = 'x'; // Erro: O tipo 'string' não pode ser atribuído ao tipo 'never'
Tipo literal	Conjunto de elemento único	type X = 'X'; type Y = 7;

TypeScript	Termo do conjunto	Exemplo
Valor atribuível a T	$\text{Valor} \in T$ (membro de)	<code>type XY = 'X' 'Y';</code> <code>const x: XY = 'X';</code>
T1 atribuível a T2	$T1 \subseteq T2$ (subconjunto de)	<code>type XY = 'X' 'Y';</code> <code>const x: XY = 'X';</code> <code>const j: XY = 'J';</code> // O tipo “J” não pode ser atribuído ao tipo ‘XY’.
T1 extends T2	$T1 \subseteq T2$ (subconjunto de)	<code>type X = 'X' extends string ? true : false;</code>
$T1 T2$	$T1 \cup T2$ (união)	<code>type XY = 'X' 'Y';</code> <code>type JK = 1 2;</code>
$T1 \& T2$	$T1 \cap T2$ (interseção)	<code>type X = { a: string }</code> <code>type Y = { b: string }</code> <code>type XY = X & Y</code> <code>const x: XY = { a: 'a', b: 'b' }</code>
unknown	Conjunto universal	<code>const x: unknown = 1</code>

Uma união, ($T1 | T2$), cria um conjunto mais amplo (ambos):

```

type X = {
  a: string;
};
type Y = {
  b: string;
};
type XY = X | Y;
const r: XY = { a: 'a', b: 'x' }; // Válido

```

Uma interseção, (T1 & T2), cria um conjunto mais estreito (apenas o que é compartilhado):

```

type X = {
  a: string;
};
type Y = {
  a: string;
  b: string;
};
type XY = X & Y;
const r: XY = { a: 'a' }; // Inválido
const j: XY = { a: 'a', b: 'b' }; // Válido

```

A palavra-chave `extends` pode ser considerada como “subconjunto de” neste contexto. Ela define uma restrição para um tipo. Quando `extends` é usado com um `generic`, ele restringe o parâmetro de tipo `generic` a um tipo mais específico.

Observe que `extends` aqui não tem nada a ver com herança de classes no sentido de OOP.

TypeScript trabalha com tipos estruturais e não possui uma hierarquia nominal rígida. Na verdade, como no exemplo abaixo, dois tipos podem se sobrepor sem que um seja um subtipo do outro, porque TypeScript considera a estrutura, ou forma, dos objetos.

```

interface X {
  a: string;
}
interface Y extends X {
  b: string;
}

```

```

}
interface Z extends Y {
  c: string;
}
const z: Z = { a: 'a', b: 'b', c: 'c' };
interface X1 {
  a: string;
}
interface Y1 {
  a: string;
  b: string;
}
interface Z1 {
  a: string;
  b: string;
  c: string;
}
const z1: Z1 = { a: 'a', b: 'b', c: 'c' };

const r: Z1 = z; // Válido

```

Atribuir um tipo: Declarações de Tipo e Asserções de Tipo

Um tipo pode ser atribuído de diferentes maneiras no TypeScript:

Declaração de Tipo

No exemplo a seguir, usamos `x: X` (“: Tipo”) para declarar um tipo para a variável `x`.

```

type X = {
  a: string;
};

// Declaração de tipo
const x: X = {
  a: 'a',
};

```

Se a variável não estiver no formato especificado, o TypeScript relatará um erro. Por exemplo:

```
type X = {  
  a: string;  
};  
  
const x: X = {  
  a: 'a',  
  b: 'b', // Erro: O objeto literal só pode especificar  
          propriedades conhecidas  
};
```

Asserção de Tipo

É possível adicionar uma asserção usando a palavra-chave `as`. Isso informa ao compilador que o desenvolvedor tem mais informações sobre um tipo e silencia quaisquer erros que possam ocorrer.

Por exemplo:

```
type X = {  
  a: string;  
};  
const x = {  
  a: 'a',  
  b: 'b',  
} as X;
```

No exemplo acima, o objeto `x` é asseverado como tendo o tipo `X` usando a palavra-chave `as`. Isso informa ao compilador TypeScript que o objeto está em conformidade com o tipo especificado, embora tenha uma propriedade `b` adicional não presente na definição do tipo.

Asserções de tipo são úteis em situações onde um tipo mais específico precisa ser especificado, especialmente ao trabalhar com o DOM. Por exemplo:

```
const myInput = document.getElementById('my_input') as
HTMLInputElement;
```

Aqui, a asserção de tipo `as HTMLInputElement` é usada para dizer ao TypeScript que o resultado de `getElementById` deve ser tratado como um `HTMLInputElement`. Asserções de tipo também podem ser usadas para mapear chaves novamente, conforme mostrado no exemplo abaixo com literais de template:

```
type J<Type> = {
  [Property in keyof Type as `prefix_${string &
    Property}`]: () => Type[Property];
};
type X = {
  a: string;
  b: number;
};
type Y = J<X>;
```

Neste exemplo, o tipo `J<Type>` usa um tipo mapeado com um literal de template para mapear as chaves de `Type`. Ele cria novas propriedades com um “`prefix_`” adicionado a cada chave, e seus valores correspondentes são funções que retornam os valores originais da propriedade.

Vale a pena notar que, ao usar uma asserção de tipo, o TypeScript não executará a verificação de excesso de propriedades. Portanto, geralmente é preferível usar uma Declaração de Tipo quando a estrutura do objeto for conhecida antecipadamente.

Declarações de Ambiente (Ambient Declarations)

Declarações de ambiente são arquivos que descrevem tipos para código JavaScript; eles têm o formato de nome de arquivo `.d.ts`. Geralmente são importados e usados para anotar bibliotecas JavaScript existentes ou para adicionar tipos a arquivos JS existentes em seu projeto.

Muitos tipos de bibliotecas comuns podem ser encontrados em:
<https://github.com/DefinitelyTyped/DefinitelyTyped/>

e podem ser instalados usando:

```
npm install --save-dev @types/nome-da-biblioteca
```

Para suas Declarações de Ambiente definidas, você pode importar usando a referência de “barra tripla”:

```
/// <reference path="./library-types.d.ts" />
```

Você pode usar Declarações de Ambiente até mesmo em arquivos JavaScript usando `// @ts-check`.

A palavra-chave `declare` habilita definições de tipo para código JavaScript existente sem importá-lo, servindo como um marcador para tipos de outro arquivo ou globalmente.

Verificação de Propriedades e Verificação de Excesso de Propriedades

O TypeScript é baseado em um sistema de tipos estrutural, mas a verificação de excesso de propriedades é um recurso do TypeScript que permite verificar se um objeto tem exatamente as propriedades especificadas no tipo.

A Verificação de Excesso de Propriedades é executada ao atribuir objetos literais a variáveis ou ao passá-los como argumentos para funções, por exemplo.

```
type X = {  
  a: string;  
};  
const y = { a: 'a', b: 'b' };
```

```
const x: X = y; // Válido por causa da tipagem estrutural
const w: X = { a: 'a', b: 'b' }; // Inválido por causa da
verificação de excesso de propriedades
```

Tipos Fracos (Weak Types)

Um tipo é considerado fraco quando não contém nada além de um conjunto de todas as propriedades opcionais:

```
type X = {
  a?: string;
  b?: string;
};
```

O TypeScript considera um erro atribuir qualquer coisa a um tipo fraco quando não há sobreposição; por exemplo, o seguinte lança um erro:

```
type Options = {
  a?: string;
  b?: string;
};
```

```
const fn = (options: Options) => undefined;
```

```
fn({ c: 'c' }); // Inválido
```

Embora não recomendado, se necessário, é possível ignorar esta verificação usando asserção de tipo:

```
type Options = {
  a?: string;
  b?: string;
};
const fn = (options: Options) => undefined;
fn({ c: 'c' } as Options); // Válido
```

Ou adicionando `unknown` à assinatura de índice do tipo fraco:

```

type Options = {
  [prop: string]: unknown;
  a?: string;
  b?: string;
};

const fn = (options: Options) => undefined;
fn({ c: 'c' }); // Válido

```

Verificação Estrita de Objeto Literal (Freshness)

A verificação estrita de objeto literal, às vezes chamada de “freshness”, é um recurso do TypeScript que ajuda a capturar propriedades em excesso ou com erro de ortografia que, de outra forma, passariam despercebidas em verificações normais de tipo estrutural.

Ao criar um objeto literal, o compilador TypeScript o considera “fresco” (fresh). Se o objeto literal for atribuído a uma variável ou passado como um parâmetro, o TypeScript lançará um erro se o objeto literal especificar propriedades que não existem no tipo de destino.

No entanto, a “freshness” desaparece quando um objeto literal é alargado ou quando uma asserção de tipo é usada.

Aqui estão alguns exemplos para ilustrar:

```

type X = { a: string };
type Y = { a: string; b: string };

let x: X;
x = { a: 'a', b: 'b' }; // Verificação de Freshness: Atribuição inválida
var y: Y;
y = { a: 'a', bx: 'bx' }; // Verificação de Freshness: Atribuição inválida

```

```
const fn = (x: X) => console.log(x.a);

fn(x);
fn(y); // Alargamento (Widening): Sem erros, estruturalmente
       compatível em termos de tipo

fn({ a: 'a', bx: 'b' }); // Verificação de Freshness: Argumento
                          inválido

let c: X = { a: 'a' };
let d: Y = { a: 'a', b: '' };
c = d; // Alargamento: Sem verificação de Freshness
```

Inferência de Tipo

O TypeScript pode inferir tipos quando nenhuma anotação é fornecida durante a:

- Inicialização da variável.
- Inicialização de membros.
- Definição de valores padrão para parâmetros.
- Tipo de retorno da função.

Por exemplo:

```
let x = 'x'; // O tipo inferido é string
```

O compilador TypeScript analisa o valor ou expressão e determina seu tipo com base nas informações disponíveis.

Inferências Mais Avançadas

Quando várias expressões são usadas na inferência de tipo, o TypeScript procura pelos “melhores tipos comuns” (best common types). Por exemplo:

```
let x = [1, 'x', 1, null]; // O tipo inferido é: (string | number | null)[]
```

Se o compilador não conseguir encontrar os melhores tipos comuns, ele retorna um tipo de união. Por exemplo:

```
let x = [new RegExp('x'), new Date()]; // O tipo inferido é: (RegExp | Date)[]
```

O TypeScript utiliza a “tipagem contextual” baseada na localização da variável para inferir tipos. No exemplo a seguir, o compilador sabe que `e` é do tipo `MouseEvent` por causa do tipo de evento `click` definido no arquivo `lib.d.ts`, que contém declarações de ambiente para várias construções JavaScript comuns e o DOM:

```
window.addEventListener('click', function (e) {}); // O tipo inferido de e é MouseEvent
```

Alargamento de Tipo (Type Widening)

O alargamento de tipo (type widening) é o processo no qual o TypeScript atribui um tipo a uma variável inicializada quando nenhuma anotação de tipo foi fornecida. Ele permite tipos de mais estreitos para mais amplos, mas não o contrário. No exemplo a seguir:

```
let x = 'x'; // O TypeScript infere como string, um tipo amplo
let y: 'y' | 'x' = 'y'; // o tipo de y é uma união de tipos literais
y = x; // Inválido: O tipo 'string' não pode ser atribuído ao tipo '"x" | "y"'.
```

O TypeScript atribui `string` a `x` com base no valor único fornecido durante a inicialização (`x`); este é um exemplo de alargamento.

O TypeScript fornece maneiras de ter controle sobre o processo de alargamento, por exemplo, usando “`const`”.

Const

O uso da palavra-chave `const` ao declarar uma variável resulta em uma inferência de tipo mais estreita no TypeScript.

Por exemplo:

```
const x = 'x'; // O TypeScript infere o tipo de x como 'x', um tipo mais estreito
let y: 'y' | 'x' = 'y';
y = x; // Válido: O tipo de x é inferido como 'x'
```

Ao usar `const` para declarar a variável `x`, seu tipo é estreitado para o valor literal específico `'x'`. Como o tipo de `x` é estreitado, ele pode ser atribuído à variável `y` sem nenhum erro. A razão pela qual o tipo pode ser inferido é porque as variáveis `const` não podem ser reatribuídas, portanto seu tipo pode ser estreitado para um tipo literal específico, neste caso, o tipo literal `'x'`.

Modificador Const em Parâmetros de Tipo

A partir da versão 5.0 do TypeScript, é possível especificar o atributo `const` em um parâmetro de tipo genérico. Isso permite inferir o tipo mais preciso possível. Vejamos um exemplo sem usar `const`:

```
function identity<T>(value: T) {
    // Sem const aqui
    return value;
}
const values = identity({ a: 'a', b: 'b' }); // O tipo inferido é:
{ a: string; b: string; }
```

Como você pode ver, as propriedades `a` e `b` são inferidas com o tipo `string`.

Agora, vejamos a diferença com a versão `const`:

```
function identity<const T>(value: T) {
    // Usando modificador const em parâmetros de tipo
```

```

    return value;
}
const values = identity({ a: 'a', b: 'b' }); // O tipo inferido é:
{ a: "a"; b: "b"; }

```

Agora podemos ver que as propriedades a e b são inferidas como const, portanto a e b são tratados como literais de string em vez de apenas tipos string.

Assertão Const (Const assertion)

Este recurso permite declarar uma variável com um tipo literal mais preciso baseado em seu valor de inicialização, sinalizando ao compilador que o valor deve ser tratado como um literal imutável. Aqui estão alguns exemplos:

Em uma única propriedade:

```

const v = {
  x: 3 as const,
};
v.x = 3;

```

Em um objeto inteiro:

```

const v = {
  x: 1,
  y: 2,
} as const;

```

Isso pode ser particularmente útil ao definir o tipo para uma tupla:

```

const x = [1, 2, 3]; // number[]
const y = [1, 2, 3] as const; // Tupla de readonly [1, 2, 3]

```

Anotação de Tipo Explícita

Podemos ser específicos e passar um tipo; no exemplo a seguir, a propriedade x é do tipo number:

```
const v = {  
  x: 1, // Tipo inferido: number (alargamento)  
};  
v.x = 3; // Válido
```

Podemos tornar a anotação de tipo mais específica usando uma união de tipos literais:

```
const v: { x: 1 | 2 | 3 } = {  
  x: 1, // x agora é uma união de tipos literais: 1 | 2 | 3  
};  
v.x = 3; // Válido  
v.x = 100; // Inválido
```

Estreitamento de Tipo (Type Narrowing)

O Estreitamento de Tipo (Type Narrowing) é o processo no TypeScript onde um tipo geral é estreitado para um tipo mais específico. Isso ocorre quando o TypeScript analisa o código e determina que certas condições ou operações podem refinar a informação do tipo.

O estreitamento de tipos pode ocorrer de diferentes maneiras, incluindo:

Condições

Ao usar instruções condicionais, como `if` ou `switch`, o TypeScript pode estreitar o tipo com base no resultado da condição. Por exemplo:

```
let x: number | undefined = 10;  
  
if (x !== undefined) {  
  x += 100; // O tipo é number, que foi estreitado pela condição  
}
```

Lançando ou retornando

Lançar um erro ou retornar cedo de uma ramificação pode ser usado para ajudar o TypeScript a estreitar um tipo. Por exemplo:

```
let x: number | undefined = 10;

if (x === undefined) {
    throw 'erro';
}
x += 100;
```

Outras formas de estreitar tipos no TypeScript incluem:

- Operador `instanceof`: Usado para verificar se um objeto é uma instância de uma classe específica.
- Operador `in`: Usado para verificar se uma propriedade existe em um objeto.
- Operador `typeof`: Usado para verificar o tipo de um valor em tempo de execução.
- Funções integradas como `Array.isArray()`: Usadas para verificar se um valor é um array.

União Discriminada

O uso de uma “União Discriminada” é um padrão no TypeScript onde uma “tag” explícita é adicionada aos objetos para distinguir entre diferentes tipos dentro de uma união. Este padrão também é conhecido como “união tagueada” (tagged union). No exemplo a seguir, a “tag” é representada pela propriedade “type”:

```
type A = { type: 'type_a'; value: number };
type B = { type: 'type_b'; value: string };

const x = (input: A | B): string | number => {
    switch (input.type) {
        case 'type_a':
            return input.value + 100; // o tipo é A
        case 'type_b':
```

```

        return input.value + 'extra'; // o tipo é B
    }
};

```

Proteções de Tipo Definidas pelo Usuário (User-Defined Type Guards)

Em casos onde o TypeScript não é capaz de determinar um tipo, é possível escrever uma função auxiliar conhecida como “proteção de tipo definida pelo usuário” (user-defined type guard). No exemplo a seguir, utilizaremos um Predicado de Tipo para estreitar o tipo após aplicar certa filtragem:

```

const data = ['a', null, 'c', 'd', null, 'f'];

const r1 = data.filter(x => x !== null); // O tipo é (string | null)
[], o TypeScript não foi capaz de inferir o tipo corretamente

const isValid = (item: string | null): item is string => item !==
null; // Protetor de tipo customizado

const r2 = data.filter(isValid); // O tipo está correto agora
string[], ao usar o protetor de tipo predicado conseguimos
estrear o tipo

```

Redução de tipos com switch-true

O TypeScript 5.3 adiciona a redução de tipos com switch-true, permitindo substituir cadeias complexas de if/else por switch (true) usando condições booleanas. Isso melhora a legibilidade e ainda reduz os tipos. É semelhante ao casamento de padrões, mas mais simples.

```

function classify(x: unknown) {
    switch (true) {
        case typeof x === 'string':
            return `${x.toUpperCase()}`;
        case typeof x === 'number':
            return x > 0 ? 'positive' : 'negative';
    }
}

```

```

        case Array.isArray(x):
            return `[${x.length} items]`;
        default:
            return 'something else';
    }
}

```

Tipos Primitivos

O TypeScript suporta 7 tipos primitivos. Um tipo de dado primitivo refere-se a um tipo que não é um objeto e não possui nenhum método associado a ele. No TypeScript, todos os tipos primitivos são imutáveis, o que significa que seus valores não podem ser alterados uma vez que são atribuídos.

string

O tipo primitivo `string` armazena dados textuais, e o valor é sempre delimitado por aspas duplas ou simples.

```

const x: string = 'x';
const y: string = 'y';

```

As strings podem abranger várias linhas se estiverem rodeadas pelo caractere de crase (```):

```

let sentence: string = `xxx,
  yyy`;

```

boolean

O tipo de dado `boolean` no TypeScript armazena um valor binário, seja `true` ou `false`.

```

const isReady: boolean = true;

```

number

Um tipo de dado `number` no TypeScript é representado com um valor de ponto flutuante de 64 bits. Um tipo `number` pode representar inteiros e frações. O TypeScript também suporta hexadecimal, binário e octal, por exemplo:

```
const decimal: number = 10;
const hexadecimal: number = 0xa00d; // Hexadecimal começa com 0x
const binary: number = 0b1010; // Binário começa com 0b
const octal: number = 0o633; // Octal começa com 0o
```

bigint

Um `bigint` representa valores numéricos muito grandes ($2^{53} - 1$) que não podem ser representados com um `number`.

Um `bigint` pode ser criado chamando a função integrada `BigInt()` ou adicionando `n` ao final de qualquer literal numérico inteiro:

```
const x: bigint = BigInt(9007199254740991);
const y: bigint = 9007199254740991n;
```

Notas:

- Valores `bigint` não podem ser misturados com `number` e não podem ser usados com a função integrada `Math`; eles devem ser coeridos para o mesmo tipo.
- Valores `bigint` estão disponíveis apenas se a configuração da meta (target) for ES2020 ou superior.

Symbol

Symbols são identificadores únicos que podem ser usados como chaves de propriedade em objetos para evitar conflitos de nomenclatura.

```

type Obj = {
  [sym: symbol]: number;
};

const a = Symbol('a');
const b = Symbol('b');
let obj: Obj = {};
obj[a] = 123;
obj[b] = 456;

console.log(obj[a]); // 123
console.log(obj[b]); // 456

```

null e undefined

Os tipos `null` e `undefined` representam a ausência de valor.

O tipo `undefined` significa que o valor não foi atribuído ou inicializado, ou indica uma ausência não intencional de valor.

O tipo `null` significa que sabemos que o campo não possui um valor, portanto o valor está indisponível; indica uma ausência intencional de valor.

Array

Um array é um tipo de dado que pode armazenar múltiplos valores do mesmo tipo ou não. Ele pode ser definido usando a seguinte sintaxe:

```

const x: string[] = ['a', 'b'];
const y: Array<string> = ['a', 'b'];
const j: Array<string | number> = ['a', 1, 'b', 2]; // União

```

O TypeScript suporta arrays somente leitura (readonly) usando a seguinte sintaxe:

```
const x: readonly string[] = ['a', 'b']; // Modificador readonly
const y: ReadonlyArray<string> = ['a', 'b'];
const j: ReadonlyArray<string | number> = ['a', 1, 'b', 2];
j.push('x'); // Inválido
```

O TypeScript suporta tupla e tupla somente leitura:

```
const x: [string, number] = ['a', 1];
const y: readonly [string, number] = ['a', 1];
```

any

O tipo de dado any representa literalmente “qualquer” valor; é o valor padrão quando o TypeScript não consegue inferir o tipo ou quando este não é especificado.

Ao usar any, o compilador TypeScript ignora a verificação de tipo, portanto não há segurança de tipo quando o any está sendo usado. Geralmente, não use any para silenciar o compilador quando ocorre um erro; em vez disso, concentre-se em corrigir o erro, pois ao usar any é possível quebrar contratos e perdemos os benefícios do preenchimento automático do TypeScript.

O tipo any pode ser útil durante uma migração gradual de JavaScript para TypeScript, pois pode silenciar o compilador.

Para novos projetos, use a configuração do TypeScript noImplicitAny, que permite que o TypeScript emita erros onde any é usado ou inferido.

O tipo any é geralmente uma fonte de erros que podem mascarar problemas reais com seus tipos. Evite usá-lo o máximo possível.

Anotações de Tipo

Em variáveis declaradas usando `var`, `let` e `const`, é possível adicionar opcionalmente um tipo:

```
const x: number = 1;
```

O TypeScript faz um bom trabalho ao inferir tipos, especialmente quando são simples, portanto essas declarações, na maioria dos casos, não são necessárias.

Em funções, é possível adicionar anotações de tipo aos parâmetros:

```
function sum(a: number, b: number) {  
    return a + b;  
}
```

O seguinte é um exemplo usando funções anônimas (as chamadas funções lambda):

```
const sum = (a: number, b: number) => a + b;
```

Essas anotações podem ser evitadas quando um valor padrão para um parâmetro está presente:

```
const sum = (a = 10, b: number) => a + b;
```

Anotações de tipo de retorno podem ser adicionadas às funções:

```
const sum = (a = 10, b: number): number => a + b;
```

Isso é útil especialmente para funções mais complexas, pois escrever explicitamente o tipo de retorno antes de uma implementação pode ajudar a pensar melhor sobre a função.

Geralmente, considere anotar as assinaturas de tipo, mas não as variáveis locais do corpo, e sempre adicione tipos a objetos literais.

Propriedades Opcionais

Um objeto pode especificar Propriedades Opcionais adicionando um ponto de interrogação ? ao final do nome da propriedade:

```
type X = {  
  a: number;  
  b?: number; // Opcional  
};
```

É possível especificar um valor padrão quando uma propriedade é opcional:

```
type X = {  
  a: number;  
  b?: number;  
};  
const x = ({ a, b = 100 }: X) => a + b;
```

Propriedades Somente Leitura (Readonly)

É possível impedir a escrita em uma propriedade usando o modificador `readonly`, que garante que a propriedade não possa ser reescrita, mas não fornece nenhuma garantia de imutabilidade total:

```
interface Y {  
  readonly a: number;  
}
```

```
type X = {  
  readonly a: number;  
};
```

```
type J = Readonly<{  
  a: number;  
}>;
```

```
type K = {  
  readonly [index: number]: string;  
};
```

Assinaturas de Índice (Index Signatures)

No TypeScript, podemos usar como assinatura de índice `string`, `number` e `symbol`:

```
type K = {  
  [name: string | number]: string;  
};  
const k: K = { x: 'x', 1: 'b' };  
console.log(k['x']);  
console.log(k[1]);  
console.log(k['1']); // Mesmo resultado que k[1]
```

Observe que o JavaScript converte automaticamente um índice com `number` em um índice com `string`, portanto `k[1]` ou `k["1"]` retornam o mesmo valor.

Estendendo Tipos

É possível estender uma interface (copiar membros de outro tipo):

```
interface X {  
  a: string;  
}  
interface Y extends X {  
  b: string;  
}
```

Também é possível estender de múltiplos tipos:

```
interface A {  
  a: string;  
}  
interface B {
```

```

    b: string;
}
interface Y extends A, B {
    y: string;
}

```

A palavra-chave `extends` funciona apenas em interfaces e classes; para tipos, use uma interseção:

```

type A = {
    a: number;
};
type B = {
    b: number;
};
type C = A & B;

```

É possível estender um tipo usando uma interface, mas não o contrário:

```

type A = {
    a: string;
};
interface B extends A {
    b: string;
}

```

Tipos Literais

Um Tipo Literal é um conjunto de elemento único a partir de um tipo coletivo; ele define um valor exato que é um primitivo do JavaScript.

Os Tipos Literais no TypeScript são números, strings e booleanos.

Exemplo de literais:

```

const a = 'a'; // Tipo literal de string
const b = 1; // Tipo literal numérico
const c = true; // Tipo literal booleano

```

Tipos Literais de String, Numéricos e Booleanos são usados em uniões, protetores de tipo (type guards) e apelidos de tipo (type aliases). No exemplo a seguir, você pode ver um apelido de tipo de união. O consiste apenas nos valores especificados; nenhuma outra string é válida:

```
type O = 'a' | 'b' | 'c';
```

Inferência Literal

A Inferência Literal é um recurso do TypeScript que permite que o tipo de uma variável ou parâmetro seja inferido com base em seu valor.

No exemplo a seguir, podemos ver que o TypeScript considera `x` um tipo literal, pois o valor não pode ser alterado posteriormente, enquanto `y` é inferido como string, pois pode ser modificado posteriormente.

```
const x = 'x'; // Tipo literal de 'x', porque este valor não pode ser alterado
let y = 'y'; // Tipo string, pois podemos alterar este valor
```

No exemplo a seguir, podemos ver que o `x` foi inferido como uma string (e não um literal de `a`), pois o TypeScript considera que o valor pode ser alterado posteriormente.

```
type X = 'a' | 'b';

let o = {
  x: 'a', // Esta é uma string mais ampla (wider string)
};

const fn = (x: X) => `${x}-foo`;

console.log(fn(o.x)); // Argument of type 'string' is not assignable to parameter of type 'X'
```

Como você pode observar, o código lança um erro ao passar `o.x` para `fn`, pois `X` é um tipo mais estreito (narrower).

Podemos resolver este problema usando asserção de tipo com `const` ou o tipo `X`:

```
let o = {  
  x: 'a' as const,  
};
```

ou:

```
let o = {  
  x: 'a' as X,  
};
```

strictNullChecks

`strictNullChecks` é uma opção do compilador TypeScript que impõe a verificação estrita de nulos. Quando esta opção está habilitada, variáveis e parâmetros só podem receber `null` | `undefined` se tiverem sido explicitamente declarados como sendo desse tipo usando o tipo de união `null` | `undefined`. Se uma variável ou parâmetro não for explicitamente declarado como anulável, o TypeScript gerará um erro para evitar possíveis erros de tempo de execução.

Enums

No TypeScript, um `enum` é um conjunto de valores constantes nomeados.

```
enum Color {  
  Red = '#ff0000',  
  Green = '#00ff00',  
  Blue = '#0000ff',  
}
```

Enums podem ser definidos de diferentes maneiras:

Enums numéricos

No TypeScript, um Enum Numérico é um Enum onde cada constante recebe um valor numérico, começando em 0 por padrão.

```
enum Size {  
    Small, // o valor começa de 0  
    Medium,  
    Large,  
}
```

É possível especificar valores personalizados atribuindo-os explicitamente:

```
enum Size {  
    Small = 10,  
    Medium,  
    Large,  
}  
console.log(Size.Medium); // 11
```

Enums de string

No TypeScript, um Enum de String é um Enum onde cada constante recebe um valor de string.

```
enum Language {  
    English = 'EN',  
    Spanish = 'ES',  
}
```

Nota: O TypeScript permite o uso de Enums heterogêneos, onde membros de string e numéricos podem coexistir.

Enums constantes

Um enum constante (const enum) no TypeScript é um tipo especial de Enum onde todos os valores são conhecidos em tempo de compilação e são inseridos diretamente (inlined) onde quer que o enum seja usado, resultando em um código mais eficiente.

```
const enum Language {  
    English = 'EN',  
    Spanish = 'ES',  
}  
console.log(Language.English);
```

Será compilado para:

```
console.log('EN' /* Language.English */);
```

Notas: Enums Constantes têm valores fixos (hardcoded), apagando o Enum, o que pode ser mais eficiente em bibliotecas autocontidas, mas geralmente não é desejável. Além disso, enums constantes não podem ter membros computados.

Mapeamento reverso

No TypeScript, os mapeamentos reversos em Enums referem-se à capacidade de recuperar o nome do membro do Enum a partir de seu valor. Por padrão, os membros do Enum têm mapeamentos diretos (forward mappings) do nome para o valor, mas mapeamentos reversos podem ser criados definindo explicitamente os valores para cada membro. Os mapeamentos reversos são úteis quando você precisa procurar um membro do Enum pelo seu valor ou quando precisa iterar sobre todos os membros do Enum. Note que apenas membros de enums numéricos gerarão mapeamentos reversos, enquanto membros de Enums de String não possuem um mapeamento reverso gerado.

O seguinte enum:

```
enum Grade {
  A = 90,
  B = 80,
  C = 70,
  F = 'fail',
}
```

Compila para:

```
'use strict';
var Grade;
(function (Grade) {
  Grade[(Grade['A'] = 90)] = 'A';
  Grade[(Grade['B'] = 80)] = 'B';
  Grade[(Grade['C'] = 70)] = 'C';
  Grade['F'] = 'fail';
})(Grade || (Grade = {}));
```

Portanto, mapear valores para chaves funciona para membros de enums numéricos, mas não para membros de enums de string:

```
enum Grade {
  A = 90,
  B = 80,
  C = 70,
  F = 'fail',
}

const myGrade = Grade.A;
console.log(Grade[myGrade]); // A
console.log(Grade[90]); // A

const failGrade = Grade.F;
console.log(failGrade); // fail
console.log(Grade[failGrade]); // Element implicitly has an 'any'
type because index expression is not of type 'number'.
```

Enums de ambiente

Um enum de ambiente no TypeScript é um tipo de Enum que é definido em um arquivo de declaração (*.d.ts) sem uma implementação associada. Ele permite definir um conjunto de

constantes nomeadas que podem ser usadas de forma segura em relação aos tipos em diferentes arquivos, sem ter que importar os detalhes da implementação em cada arquivo.

Membros computados e constantes

No TypeScript, um membro computado é um membro de um Enum que possui um valor calculado em tempo de execução, enquanto um membro constante é um membro cujo valor é definido em tempo de compilação e não pode ser alterado durante o tempo de execução. Membros computados são permitidos em Enums regulares, enquanto membros constantes são permitidos tanto em enums regulares quanto em enums constantes (const enums).

```
// Membros constantes
enum Color {
    Red = 1,
    Green = 5,
    Blue = Red + Green,
}
console.log(Color.Blue); // geração 6 em tempo de compilação

// Membros computados
enum Color {
    Red = 1,
    Green = Math.pow(2, 2),
    Blue = Math.floor(Math.random() * 3) + 1,
}
console.log(Color.Blue); // número aleatório gerado em tempo de execução
```

Enums são representados por uniões compostas pelos tipos de seus membros. Os valores de cada membro podem ser determinados por meio de expressões constantes ou não constantes, com membros que possuem valores constantes recebendo tipos literais. Para ilustrar, considere a declaração do tipo E e seus subtipos E.A, E.B e E.C. Neste caso, E representa a união E.A | E.B | E.C.

```
const identity = (value: number) => value;
```

```
enum E {  
    A = 2 * 5, // Literal numérico  
    B = 'bar', // Literal de string  
    C = identity(42), // Computado opaco  
}
```

```
console.log(E.C); //42
```

Estreitamento (Narrowing)

O estreitamento (narrowing) no TypeScript é o processo de refinar o tipo de uma variável dentro de um bloco condicional. Isso é útil ao trabalhar com tipos de união, onde uma variável pode ter mais de um tipo.

O TypeScript reconhece várias maneiras de estreitar o tipo:

typeof type guards

O protetor de tipo (type guard) `typeof` é um protetor de tipo específico no TypeScript que verifica o tipo de uma variável com base em seu tipo JavaScript integrado.

```
const fn = (x: number | string) => {  
    if (typeof x === 'number') {  
        return x + 1; // x é number  
    }  
    return -1;  
};
```

Estreitamento de veracidade (Truthiness narrowing)

O estreitamento de veracidade (truthiness narrowing) no TypeScript funciona verificando se uma variável é verdadeira (truthy) ou falsa (falsy) para estreitar seu tipo adequadamente.

```
const toUpperCase = (name: string | null) => {  
  if (name) {  
    return name.toUpperCase();  
  } else {  
    return null;  
  }  
};
```

Estreitamento de igualdade (Equality narrowing)

O estreitamento de igualdade (equality narrowing) no TypeScript funciona verificando se uma variável é igual a um valor específico ou não, para estreitar seu tipo adequadamente.

É usado em conjunto com instruções `switch` e operadores de igualdade como `===`, `!==`, `==` e `!=` para estreitar os tipos.

```
const checkStatus = (status: 'success' | 'error') => {  
  switch (status) {  
    case 'success':  
      return true;  
    case 'error':  
      return null;  
  }  
};
```

Estreitamento com operador In

O estreitamento com o operador `in` no TypeScript é uma forma de estreitar o tipo de uma variável com base na existência de uma propriedade dentro do tipo da variável.

```
type Dog = {
  name: string;
  breed: string;
};

type Cat = {
  name: string;
  likesCream: boolean;
};

const getAnimalType = (pet: Dog | Cat) => {
  if ('breed' in pet) {
    return 'dog';
  } else {
    return 'cat';
  }
};
```

Estreitamento com instanceof

O estreitamento com o operador `instanceof` no TypeScript é uma forma de estreitar o tipo de uma variável com base em sua função construtora, verificando se um objeto é uma instância de uma determinada classe ou interface.

```
class Square {
  constructor(public width: number) {}
}

class Rectangle {
  constructor(
    public width: number,
    public height: number
  ) {}
}
```

```
function area(shape: Square | Rectangle) {
    if (shape instanceof Square) {
        return shape.width * shape.width;
    } else {
        return shape.width * shape.height;
    }
}
const square = new Square(5);
const rectangle = new Rectangle(5, 10);
console.log(area(square)); // 25
console.log(area(rectangle)); // 50
```

Atribuições

O estreitamento do TypeScript usando atribuições é uma forma de estreitar o tipo de uma variável com base no valor atribuído a ela. Quando uma variável recebe um valor, o TypeScript infere seu tipo com base no valor atribuído e estreita o tipo da variável para corresponder ao tipo inferido.

```
let value: string | number;
value = 'hello';
if (typeof value === 'string') {
    console.log(value.toUpperCase());
}
value = 42;
if (typeof value === 'number') {
    console.log(value.toFixed(2));
}
```

Análise de Fluxo de Controle

A Análise de Fluxo de Controle (Control Flow Analysis) no TypeScript é uma forma de analisar estaticamente o fluxo do código para inferir os tipos das variáveis, permitindo que o compilador estreite os tipos dessas variáveis conforme necessário, com base nos resultados da análise.

Antes do TypeScript 4.4, a análise de fluxo de código só seria aplicada ao código dentro de uma instrução `if`, mas a partir do TypeScript 4.4, ela também pode ser aplicada a expressões condicionais e acessos a propriedades discriminantes referenciados indiretamente por meio de variáveis `const`.

Por exemplo:

```
const f1 = (x: unknown) => {
  const isString = typeof x === 'string';
  if (isString) {
    x.length;
  }
};

const f2 = (
  obj: { kind: 'foo'; foo: string } | { kind: 'bar'; bar: number }
) => {
  const isFoo = obj.kind === 'foo';
  if (isFoo) {
    obj.foo;
  } else {
    obj.bar;
  }
};
```

Alguns exemplos onde o estreitamento não ocorre:

```
const f1 = (x: unknown) => {
  let isString = typeof x === 'string';
  if (isString) {
    x.length; // Erro, sem estreitamento porque isString não é const
  }
};

const f6 = (
  obj: { kind: 'foo'; foo: string } | { kind: 'bar'; bar: number }
) => {
  const isFoo = obj.kind === 'foo';
```

```

    obj = obj;
    if (isFoo) {
        obj.foo; // Erro, sem estreitamento porque obj é atribuído
no corpo da função
    }
};

```

Notas: Até cinco níveis de indireção são analisados em expressões condicionais.

Predicados de Tipo

Predicados de Tipo (Type Predicates) no TypeScript são funções que retornam um valor booleano e são usadas para estreitar o tipo de uma variável para um tipo mais específico.

```

const isString = (value: unknown): value is string => typeof value
=== 'string';

```

```

const foo = (bar: unknown) => {
    if (isString(bar)) {
        console.log(bar.toUpperCase());
    } else {
        console.log('não é uma string');
    }
};

```

O TypeScript 5.5 infere automaticamente predicados de tipo (como `x is T`) em funções como `.filter`, de forma que ele saiba quando valores como `undefined` são removidos — resultando em tipos mais precisos e menos erros; isso funciona para verificações claras (por exemplo, `x !== undefined`), mas não para verificações ambíguas como `!!x`.

```

const nums = [1, null, 2].filter(x => x !== null);

```

Unões Discriminadas

Unões Discriminadas (Discriminated Unions) no TypeScript são um tipo de união que usa uma propriedade comum, conhecida como discriminante, para estreitar o conjunto de tipos possíveis para a união.

```
type Square = {
  kind: 'square'; // Discriminante
  size: number;
};

type Circle = {
  kind: 'circle'; // Discriminante
  radius: number;
};

type Shape = Square | Circle;

const area = (shape: Shape) => {
  switch (shape.kind) {
    case 'square':
      return Math.pow(shape.size, 2);
    case 'circle':
      return Math.PI * Math.pow(shape.radius, 2);
  }
};

const square: Square = { kind: 'square', size: 5 };
const circle: Circle = { kind: 'circle', radius: 2 };

console.log(area(square)); // 25
console.log(area(circle)); // 12.566370614359172
```

O tipo never

Quando uma variável é estreitada para um tipo que não pode conter nenhum valor, o compilador TypeScript inferirá que a variável deve

ser do tipo `never`. Isso ocorre porque o Tipo `Never` representa um valor que nunca pode ser produzido.

```
const printValue = (val: string | number) => {
  if (typeof val === 'string') {
    console.log(val.toUpperCase());
  } else if (typeof val === 'number') {
    console.log(val.toFixed(2));
  } else {
    // val tem o tipo never aqui porque nunca pode ser nada
    // além de uma string ou um número
    const neverVal: never = val;
    console.log(`Valor inesperado: ${neverVal}`);
  }
};
```

Verificação de exaustividade

A verificação de exaustividade (exhaustiveness checking) é um recurso no TypeScript que garante que todos os casos possíveis de uma união discriminada sejam tratados em uma instrução `switch` ou `if`.

```
type Direction = 'up' | 'down';

const move = (direction: Direction) => {
  switch (direction) {
    case 'up':
      console.log('Movendo para cima');
      break;
    case 'down':
      console.log('Movendo para baixo');
      break;
    default:
      const exhaustiveCheck: never = direction;
      console.log(exhaustiveCheck); // Esta linha nunca será
    executada
  }
};
```

O tipo `never` é usado para garantir que o caso `default` seja exaustivo e que o TypeScript aponte um erro se um novo valor for adicionado ao tipo `Direction` sem ser tratado na instrução `switch`.

Tipos de Objeto

No TypeScript, os tipos de objeto descrevem a forma de um objeto. Eles especificam os nomes e tipos das propriedades do objeto, bem como se essas propriedades são obrigatórias ou opcionais.

No TypeScript, você pode definir tipos de objeto de duas maneiras principais:

Interface, que define a forma de um objeto especificando os nomes, tipos e a opcionalidade de suas propriedades.

```
interface User {  
    name: string;  
    age: number;  
    email?: string;  
}
```

Apelido de tipo (type alias), semelhante a uma interface, define a forma de um objeto. No entanto, ele também pode criar um novo tipo personalizado baseado em um tipo existente ou em uma combinação de tipos existentes. Isso inclui definir tipos de união, tipos de interseção e outros tipos complexos.

```
type Point = {  
    x: number;  
    y: number;  
};
```

Também é possível definir um tipo anonimamente:

```
const sum = (x: { a: number; b: number }) => x.a + x.b;  
console.log(sum({ a: 5, b: 1 }));
```

Tipo Tupla (Anônimo)

Um Tipo Tupla (Tuple Type) é um tipo que representa um array com um número fixo de elementos e seus tipos correspondentes. Um tipo tupla impõe um número específico de elementos e seus respectivos tipos em uma ordem fixa. Os tipos tupla são úteis quando você deseja representar uma coleção de valores com tipos específicos, onde a posição de cada elemento no array tem um significado específico.

```
type Point = [number, number];
```

Tipo Tupla Nomeado (Rotulado)

Os tipos tupla podem incluir rótulos (labels) ou nomes opcionais para cada elemento. Esses rótulos são para legibilidade e assistência de ferramentas, e não afetam as operações que você pode realizar com eles.

```
type T = string;  
type Tuple1 = [T, T];  
type Tuple2 = [a: T, b: T];  
type Tuple3 = [a: T, T]; // Tupla Nomeada mais Tupla Anônima
```

Tupla de Comprimento Fixo

Uma Tupla de Comprimento Fixo é um tipo específico de tupla que impõe um número fixo de elementos de tipos específicos e proíbe quaisquer modificações no comprimento da tupla uma vez definida.

Tuplas de Comprimento Fixo são úteis quando você precisa representar uma coleção de valores com um número específico de elementos e tipos específicos, e deseja garantir que o comprimento e os tipos da tupla não possam ser alterados inadvertidamente.

```
const x = [10, 'hello'] as const;  
x.push(2); // Erro
```

Tipo União

Um Tipo União (Union Type) é um tipo que representa um valor que pode ser um de vários tipos. Tipos União são denotados usando o símbolo `|` entre cada tipo possível.

```
let x: string | number;  
x = 'hello'; // Válido  
x = 123; // Válido
```

Tipos de Interseção

Um Tipo de Interseção (Intersection Type) é um tipo que representa um valor que possui todas as propriedades de dois ou mais tipos. Tipos de Interseção são denotados usando o símbolo `&` entre cada tipo.

```
type X = {  
  a: string;  
};  
  
type Y = {  
  b: string;  
};  
  
type J = X & Y; // Interseção  
  
const j: J = {  
  a: 'a',  
  b: 'b',  
};
```

Indexação de Tipo

Indexação de tipo (type indexing) refere-se à capacidade de definir tipos que podem ser indexados por uma chave não conhecida antecipadamente, usando uma assinatura de índice para especificar o tipo para propriedades que não são declaradas explicitamente.

```
type Dictionary<T> = {  
  [key: string]: T;  
};  
const myDict: Dictionary<string> = { a: 'a', b: 'b' };  
console.log(myDict['a']); // Retorna a
```

Tipo a partir de Valor

Tipo a partir de Valor (Type from Value) no TypeScript refere-se à inferência automática de um tipo a partir de um valor ou expressão através da inferência de tipos.

```
const x = 'x'; // O TypeScript infere 'x' como um literal de string  
com 'const' (imutável), mas alarga para 'string' com 'let'  
(atribuível novamente).
```

Tipo a partir de Retorno de Função

Tipo a partir de Retorno de Função refere-se à capacidade de inferir automaticamente o tipo de retorno de uma função com base em sua implementação. Isso permite que o TypeScript determine o tipo do valor retornado pela função sem anotações de tipo explícitas.

```
const add = (x: number, y: number) => x + y; // O TypeScript pode  
inferir que o tipo de retorno da função é um número
```

Tipo a partir de Módulo

Tipo a partir de Módulo refere-se à capacidade de usar os valores exportados de um módulo para inferir automaticamente seus tipos. Quando um módulo exporta um valor com um tipo específico, o TypeScript pode usar essa informação para inferir automaticamente o tipo desse valor quando ele é importado para outro módulo.

```
// calc.ts
export const add = (x: number, y: number) => x + y;
// index.ts
import { add } from 'calc';
const r = add(1, 2); // r é number
```

Tipos Mapeados

Tipos Mapeados (Mapped Types) no TypeScript permitem criar novos tipos baseados em um tipo existente, transformando cada propriedade por meio de uma função de mapeamento. Ao mapear tipos existentes, você pode criar novos tipos que representam a mesma informação em um formato diferente. Para criar um tipo mapeado, você acessa as propriedades de um tipo existente usando o operador `keyof` e as altera para produzir um novo tipo. No exemplo a seguir:

```
type MyMappedType<T> = {
  [P in keyof T]: T[P] [];
};
type MyType = {
  foo: string;
  bar: number;
};
type MyNewType = MyMappedType<MyType>;
const x: MyNewType = {
  foo: ['hello', 'world'],
  bar: [1, 2, 3],
};
```

definimos `MyMappedType` para mapear sobre as propriedades de `T`, criando um novo tipo com cada propriedade sendo um array de seu tipo original. Usando isso, criamos `MyNewType` para representar a mesma informação que `MyType`, mas com cada propriedade como um array.

Modificadores de Tipos Mapeados

Os Modificadores de Tipos Mapeados no TypeScript permitem a transformação de propriedades dentro de um tipo existente:

- `readonly` ou `+readonly`: Torna uma propriedade no tipo mapeado como somente leitura.
- `-readonly`: Permite que uma propriedade no tipo mapeado seja mutável.
- `?`: Designa uma propriedade no tipo mapeado como opcional.

Exemplos:

```
type Readonly<T> = { readonly [P in keyof T]: T[P] }; // Todas as  
propriedades marcadas como somente leitura
```

```
type Mutable<T> = { -readonly [P in keyof T]: T[P] }; // Todas as  
propriedades marcadas como mutáveis
```

```
type MyPartial<T> = { [P in keyof T]?: T[P] }; // Todas as  
propriedades marcadas como opcionais
```

Tipos Condicionais (Conditional Types)

Tipos Condicionais são uma forma de criar um tipo que depende de uma condição, onde o tipo a ser criado é determinado com base no resultado da condição. Eles são definidos usando a palavra-chave `extends` e um operador ternário para escolher condicionalmente entre dois tipos.

```

type IsArray<T> = T extends any[] ? true : false;

const myArray = [1, 2, 3];
const myNumber = 42;

type IsMyArrayAnArray = IsArray<typeof myArray>; // Tipo true
type IsMyNumberAnArray = IsArray<typeof myNumber>; // Tipo false

```

Tipos Condicionais Distributivos

Tipos Condicionais Distributivos são um recurso que permite que um tipo seja distribuído sobre uma união de tipos, aplicando uma transformação a cada membro da união individualmente. Isso pode ser especialmente útil ao trabalhar com tipos mapeados ou tipos de ordem superior.

```

type Nullable<T> = T extends any ? T | null : never;
type NumberOrBool = number | boolean;
type NullableNumberOrBool = Nullable<NumberOrBool>; // number |
boolean | null

```

infer Inferência de Tipo em Tipos Condicionais

A palavra-chave `infer` é usada em tipos condicionais para inferir (extrair) o tipo de um parâmetro genérico de um tipo que depende dele. Isso permite escrever definições de tipo mais flexíveis e reutilizáveis.

```

type ElementType<T> = T extends (infer U)[] ? U : never;
type Numbers = ElementType<number[]>; // number
type Strings = ElementType<string[]>; // string

```

Tipos Condicionais Predefinidos

No TypeScript, os Tipos Condicionais Predefinidos são tipos condicionais integrados fornecidos pela linguagem. Eles são projetados para realizar transformações comuns de tipo com base nas características de um determinado tipo.

`Exclude<UnionType, ExcludedType>`: Este tipo remove todos os tipos de `Type` que são atribuíveis a `ExcludedType`.

`Extract<Type, Union>`: Este tipo extrai todos os tipos de `Union` que são atribuíveis a `Type`.

`NonNullable<Type>`: Este tipo remove `null` e `undefined` de `Type`.

`ReturnType<Type>`: Este tipo extrai o tipo de retorno de uma função `Type`.

`Parameters<Type>`: Este tipo extrai os tipos de parâmetros de uma função `Type`.

`Required<Type>`: Este tipo torna todas as propriedades em `Type` obrigatórias.

`Partial<Type>`: Este tipo torna todas as propriedades em `Type` opcionais.

`Readonly<Type>`: Este tipo torna todas as propriedades em `Type` somente leitura (`readonly`).

Tipos de União de Template (Template Union Types)

Tipos de união de template podem ser usados para mesclar e manipular texto dentro do sistema de tipos, por exemplo:

```
type Status = 'active' | 'inactive';
type Products = 'p1' | 'p2';
type ProductId = `id-${Products}-${Status}`; // "id-p1-active" |
" id-p1-inactive" | "id-p2-active" | "id-p2-inactive"
```

Tipo any

O tipo `any` é um tipo especial (supertipo universal) que pode ser usado para representar qualquer tipo de valor (primitivos, objetos, arrays, funções, erros, símbolos). É frequentemente usado em situações onde o tipo de um valor não é conhecido em tempo de compilação, ou ao trabalhar com valores de APIs externas ou bibliotecas que não possuem tipagens TypeScript.

Ao utilizar o tipo `any`, você está indicando ao compilador TypeScript que os valores devem ser representados sem quaisquer limitações. Para maximizar a segurança de tipo em seu código, considere o seguinte:

- Limite o uso de `any` a casos específicos onde o tipo é verdadeiramente desconhecido.
- Não retorne tipos `any` de uma função, pois isso enfraquece a segurança de tipo no código que a utiliza.
- Em vez de `any`, use `@ts-ignore` se precisar silenciar o compilador.

```
let value: any;
value = true; // Válido
value = 7; // Válido
```

Tipo unknown

No TypeScript, o tipo `unknown` representa um valor que é de um tipo desconhecido. Ao contrário do tipo `any`, que permite qualquer tipo de valor, o `unknown` exige uma verificação de tipo ou asserção antes de poder ser usado de uma maneira específica, portanto nenhuma

operação é permitida em um `unknown` sem primeiro asseverar ou estreitar para um tipo mais específico.

O tipo `unknown` só é atribuível a si mesmo e ao tipo `any`; é uma alternativa segura em termos de tipos ao `any`.

```
let value: unknown;

let value1: unknown = value; // Válido
let value2: any = value; // Válido
let value3: boolean = value; // Inválido
let value4: number = value; // Inválido

const add = (a: unknown, b: unknown): number | undefined =>
  typeof a === 'number' && typeof b === 'number' ? a + b :
  undefined;
console.log(add(1, 2)); // 3
console.log(add('x', 2)); // undefined
```

Tipo void

O tipo `void` é usado para indicar que uma função não retorna um valor.

```
const sayHello = (): void => {
  console.log('Hello!');
};
```

Tipo never

O tipo `never` representa valores que nunca ocorrem. É usado para denotar funções ou expressões que nunca retornam ou que lançam um erro.

Por exemplo, um loop infinito:

```
const infiniteLoop = (): never => {
    while (true) {
        // faz algo
    }
};
```

Lançando um erro:

```
const throwError = (message: string): never => {
    throw new Error(message);
};
```

O tipo `never` é útil para garantir a segurança de tipos e capturar possíveis erros em seu código. Ele ajuda o TypeScript a analisar e inferir tipos mais precisos quando usado em combinação com outros tipos e instruções de fluxo de controle, por exemplo:

```
type Direction = 'up' | 'down';
const move = (direction: Direction): void => {
    switch (direction) {
        case 'up':
            // move para cima
            break;
        case 'down':
            // move para baixo
            break;
        default:
            const exhaustiveCheck: never = direction;
            throw new Error(`Direção não tratada: ${exhaustiveCheck}`);
    }
};
```

Interface e Type

Sintaxe Comum

No TypeScript, as interfaces definem a estrutura de objetos, especificando os nomes e tipos de propriedades ou métodos que um

objeto deve possuir. A sintaxe comum para definir uma interface no TypeScript é a seguinte:

```
interface InterfaceName {  
    property1: Type1;  
    // ...  
    method1(arg1: ArgType1, arg2: ArgType2): ReturnType;  
    // ...  
}
```

Da mesma forma para a definição de tipo (type):

```
type TypeName = {  
    property1: Type1;  
    // ...  
    method1(arg1: ArgType1, arg2: ArgType2): ReturnType;  
    // ...  
};
```

interface InterfaceName ou type TypeName: Define o nome da interface/tipo. property1: Type1: Especifica as propriedades da interface junto com seus tipos correspondentes. Múltiplas propriedades podem ser definidas, cada uma separada por um ponto e vírgula. method1(arg1: ArgType1, arg2: ArgType2): ReturnType;: Especifica os métodos da interface. Métodos são definidos com seus nomes, seguidos por uma lista de parâmetros entre parênteses e o tipo de retorno. Múltiplos métodos podem ser definidas, cada uma separada por um ponto e vírgula.

Exemplo de interface:

```
interface Person {  
    name: string;  
    age: number;  
    greet(): void;  
}
```

Exemplo de tipo:

```
type TypeName = {  
    property1: string;
```

```
    method1(arg1: string, arg2: string): string;
};
```

No TypeScript, os tipos são usados para definir a forma dos dados e impor a verificação de tipos. Existem várias sintaxes comuns para definir tipos, dependendo do caso de uso específico. Aqui estão alguns exemplos:

Tipos Básicos

```
let myNumber: number = 123; // tipo number
let myBoolean: boolean = true; // tipo boolean
let myArray: string[] = ['a', 'b']; // array de strings
let myTuple: [string, number] = ['a', 123]; // tupla
```

Objetos e Interfaces

```
const x: { name: string; age: number } = { name: 'Simon', age: 7 };
```

Tipos União e Interseção

```
type MyType = string | number; // Tipo União (Union type)
let myUnion: MyType = 'hello'; // Pode ser uma string
myUnion = 123; // Ou um número

type TypeA = { name: string };
type TypeB = { age: number };
type CombinedType = TypeA & TypeB; // Tipo Interseção (Intersection type)
let myCombined: CombinedType = { name: 'John', age: 25 }; // Objeto com as propriedades name e age
```

Tipos Primitivos Integrados

O TypeScript possui vários tipos primitivos integrados que podem ser usados para definir variáveis, parâmetros de função e tipos de retorno:

- `number`: Representa valores numéricos, incluindo inteiros e números de ponto flutuante.
- `string`: Representa dados textuais.
- `boolean`: Representa valores lógicos, que podem ser `true` ou `false`.
- `null`: Representa a ausência de um valor.
- `undefined`: Representa um valor que não foi atribuído ou não foi definido.
- `symbol`: Representa um identificador único. Symbols são normalmente usados como chaves para propriedades de objetos.
- `bigint`: Representa inteiros de precisão arbitrária.
- `any`: Representa um tipo dinâmico ou desconhecido. Variáveis do tipo `any` podem conter valores de qualquer tipo e ignoram a verificação de tipos.
- `void`: Representa a ausência de qualquer tipo. É comumente usado como o tipo de retorno de funções que não retornam um valor.
- `never`: Representa um tipo para valores que nunca ocorrem. É normalmente usado como o tipo de retorno de funções que lançam um erro ou entram em um loop infinito.

Objetos JS Integrados Comuns

O TypeScript é um superconjunto do JavaScript e inclui todos os objetos integrados do JavaScript comumente usados. Você pode encontrar uma lista extensa desses objetos no site de documentação da Mozilla Developer Network (MDN):

https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Global_Objects

Aqui está uma lista de alguns objetos integrados do JavaScript comumente usados:

- Function
- Object
- Boolean
- Error
- Number
- BigInt
- Math
- Date
- String
- RegExp
- Array
- Map
- Set
- Promise
- Intl

Sobrecargas (Overloads)

As sobrecargas de função (function overloads) no TypeScript permitem definir múltiplas assinaturas para um mesmo nome de função, permitindo que as funções sejam chamadas de diversas maneiras. Aqui está um exemplo:

```
// Sobrecargas
function sayHi(name: string): string;
function sayHi(names: string[]): string[];

// Implementação
function sayHi(name: unknown): unknown {
  if (typeof name === 'string') {
    return `Hi, ${name}!`;
  } else if (Array.isArray(name)) {
    return name.map(name => `Hi, ${name}!`);
  }
}
```

```
        throw new Error('Valor inválido');
    }
```

```
sayHi('xx'); // Válido
sayHi(['aa', 'bb']); // Válido
```

Aqui está outro exemplo de uso de sobrecargas de função dentro de uma class:

```
class Greeter {
    message: string;

    constructor(message: string) {
        this.message = message;
    }

    // sobrecarga
    sayHi(name: string): string;
    sayHi(names: string[]): ReadonlyArray<string>;

    // implementação
    sayHi(name: unknown): unknown {
        if (typeof name === 'string') {
            return `${this.message}, ${name}!`;
        } else if (Array.isArray(name)) {
            return name.map(name => `${this.message}, ${name}!`);
        }
        throw new Error('o valor é inválido');
    }
}

console.log(new Greeter('Hello').sayHi('Simon'));
```

Mesclagem e Extensão

Mesclagem (merging) e extensão (extension) referem-se a dois conceitos diferentes relacionados ao trabalho com tipos e interfaces.

A mesclagem permite combinar várias declarações do mesmo nome em uma única definição, por exemplo, quando você define uma interface com o mesmo nome várias vezes:

```
interface X {  
  a: string;  
}
```

```
interface X {  
  b: number;  
}
```

```
const person: X = {  
  a: 'a',  
  b: 7,  
};
```

A extensão refere-se à capacidade de estender ou herdar de tipos ou interfaces existentes para criar novos. É um mecanismo para adicionar propriedades ou métodos adicionais a um tipo existente sem modificar sua definição original. Exemplo:

```
interface Animal {  
  name: string;  
  eat(): void;  
}
```

```
interface Bird extends Animal {  
  sing(): void;  
}
```

```
const dog: Bird = {  
  name: 'Bird 1',  
  eat() {  
    console.log('Comendo');  
  },  
  sing() {  
    console.log('Cantando');  
  },  
};
```

Diferenças entre Type e Interface

Mesclagem de declarações (aumento):

As interfaces suportam a mesclagem de declarações, o que significa que você pode definir várias interfaces com o mesmo nome, e o TypeScript as mesclará em uma única interface com as propriedades e métodos combinados. Por outro lado, os tipos (types) não suportam a mesclagem de declarações. Isso pode ser útil quando você deseja adicionar funcionalidades extras ou personalizar tipos existentes sem modificar as definições originais ou corrigir tipos ausentes ou incorretos.

```
interface A {  
    x: string;  
}  
interface A {  
    y: string;  
}  
const j: A = {  
    x: 'xx',  
    y: 'yy',  
};
```

Estendendo outros tipos/interfaces:

Tanto tipos quanto interfaces podem estender outros tipos/interfaces, mas a sintaxe é diferente. Com as interfaces, você usa a palavra-chave `extends` para herdar propriedades e métodos de outras interfaces. No entanto, uma interface não pode estender um tipo complexo, como um tipo união.

```
interface A {  
    x: string;  
    y: number;  
}  
interface B extends A {  
    z: string;  
}  
const car: B = {  
    x: 'x',  
    y: 123,  
    z: 'z',  
};
```

Para tipos, você usa o operador & para combinar múltiplos tipos em um único tipo (interseção).

```
interface A {  
  x: string;  
  y: number;  
}
```

```
type B = A & {  
  j: string;  
};
```

```
const c: B = {  
  x: 'x',  
  y: 123,  
  j: 'j',  
};
```

Tipos União e Interseção:

Os tipos (types) são mais flexíveis quando se trata de definir Tipos União e Interseção. Com a palavra-chave `type`, você pode criar facilmente tipos união usando o operador `|` e tipos interseção usando o operador `&`. Embora as interfaces também possam representar tipos união indiretamente, elas não possuem suporte integrado para tipos interseção.

```
type Department = 'dep-x' | 'dep-y'; // União
```

```
type Person = {  
  name: string;  
  age: number;  
};
```

```
type Employee = {  
  id: number;  
  department: Department;  
};
```

```
type EmployeeInfo = Person & Employee; // Interseção
```

Exemplo com interfaces:

```
interface A {  
    x: 'x';  
}  
interface B {  
    y: 'y';  
}  
  
type C = A | B; // União de interfaces
```

Classes

Sintaxe Comum de Classes

A palavra-chave `class` é usada no TypeScript para definir uma classe. Abaixo, você pode ver um exemplo:

```
class Person {  
    private name: string;  
    private age: number;  
    constructor(name: string, age: number) {  
        this.name = name;  
        this.age = age;  
    }  
    public sayHi(): void {  
        console.log(  
            `Olá, meu nome é ${this.name} e eu tenho ${this.age}  
anos.`  
        );  
    }  
}
```

A palavra-chave `class` é usada para definir uma classe chamada “Person”.

A classe possui duas propriedades privadas: `name` do tipo `string` e `age` do tipo `number`.

O construtor é definido usando a palavra-chave `constructor`. Ele recebe `name` e `age` como parâmetros e os atribui às propriedades correspondentes.

A classe possui um método `public` chamado `sayHi` que registra uma mensagem de saudação.

Para criar uma instância de uma classe no TypeScript, você pode usar a palavra-chave `new` seguida pelo nome da classe, seguida de parênteses `()`. Por exemplo:

```
const myObject = new Person('John Doe', 25);
myObject.sayHi(); // Saída: Olá, meu nome é John Doe e eu tenho 25 anos.
```

Construtor

Os construtores são métodos especiais dentro de uma classe que são usados para inicializar as propriedades do objeto quando uma instância da classe é criada.

```
class Person {
  public name: string;
  public age: number;

  constructor(name: string, age: number) {
    this.name = name;
    this.age = age;
  }

  sayHello() {
    console.log(
      `Olá, meu nome é ${this.name} e eu tenho ${this.age} anos.`
    );
  }
}
```

```
const john = new Person('Simon', 17);
john.sayHello();
```

É possível sobrecarregar um construtor usando a seguinte sintaxe:

```
type Sex = 'm' | 'f';
```

```
class Person {
  name: string;
  age: number;
  sex: Sex;

  constructor(name: string, age: number, sex?: Sex);
  constructor(name: string, age: number, sex: Sex) {
    this.name = name;
    this.age = age;
    this.sex = sex ?? 'm';
  }
}
```

```
const p1 = new Person('Simon', 17);
const p2 = new Person('Alice', 22, 'f');
```

No TypeScript, é possível definir múltiplas sobrecargas de construtor, mas você pode ter apenas uma implementação que deve ser compatível com todas as sobrecargas, o que pode ser alcançado usando um parâmetro opcional.

```
class Person {
  name: string;
  age: number;

  constructor();
  constructor(name: string);
  constructor(name: string, age: number);
  constructor(name?: string, age?: number) {
    this.name = name ?? 'Desconhecido';
    this.age = age ?? 0;
  }

  displayInfo() {
    console.log(`Nome: ${this.name}, Idade: ${this.age}`);
  }
}
```

```
}  
}
```

```
const person1 = new Person();  
person1.displayInfo(); // Nome: Desconhecido, Idade: 0
```

```
const person2 = new Person('John');  
person2.displayInfo(); // Nome: John, Idade: 0
```

```
const person3 = new Person('Jane', 25);  
person3.displayInfo(); // Nome: Jane, Idade: 25
```

Construtores Privados e Protegidos

No TypeScript, os construtores podem ser marcados como privados ou protegidos, o que restringe sua acessibilidade e uso.

Construtores Privados (Private Constructors): Podem ser chamados apenas dentro da própria classe. Construtores privados são frequentemente usados em cenários onde você deseja impor um padrão singleton ou restringir a criação de instâncias a um método factual dentro da classe.

Construtores Protegidos (Protected Constructors): Construtores protegidos são úteis quando você deseja criar uma classe base que não deve ser instanciada diretamente, mas pode ser estendida por subclasses.

```
class BaseClass {  
    protected constructor() {}  
}  
  
class DerivedClass extends BaseClass {  
    private value: number;  
  
    constructor(value: number) {  
        super();  
        this.value = value;  
    }  
}
```

```
}
```

```
// Tentar instanciar a classe base diretamente resultará em erro  
// const baseObj = new BaseClass(); // Erro: O construtor da classe  
'BaseClass' é protegido.
```

```
// Criar uma instância da classe derivada  
const derivedObj = new DerivedClass(10);
```

Modificadores de Acesso

Modificadores de Acesso `private`, `protected` e `public` são usados para controlar a visibilidade e acessibilidade dos membros da classe, como propriedades e métodos, em classes TypeScript. Esses modificadores são essenciais para impor o encapsulamento e estabelecer limites para acessar e modificar o estado interno de uma classe.

O modificador `private` restringe o acesso ao membro da classe apenas dentro da classe que o contém.

O modificador `protected` permite o acesso ao membro da classe dentro da classe que o contém e suas classes derivadas.

O modificador `public` fornece acesso irrestrito ao membro da classe, permitindo que ele seja acessado de qualquer lugar.

Get e Set

Getters e setters são métodos especiais que permitem definir comportamentos personalizados de acesso e modificação para propriedades de classe. Eles permitem encapsular o estado interno de um objeto e fornecer lógica adicional ao obter ou definir os valores das propriedades. No TypeScript, os getters e setters são definidos usando as palavras-chave `get` e `set`, respectivamente. Aqui está um exemplo:

```

class MyClass {
    private _myProperty: string;

    constructor(value: string) {
        this._myProperty = value;
    }
    get myProperty(): string {
        return this._myProperty;
    }
    set myProperty(value: string) {
        this._myProperty = value;
    }
}

```

Auto-acessores em Classes

O TypeScript versão 4.9 adiciona suporte para auto-acessores (auto-accessors), um recurso futuro do ECMAScript. Eles se assemelham a propriedades de classe, mas são declarados com a palavra-chave “accessor”.

```

class Animal {
    accessor name: string;

    constructor(name: string) {
        this.name = name;
    }
}

```

Os auto-acessores são transformados em acessores get e set privados, operando em uma propriedade inacessível.

```

class Animal {
    #__name: string;

    get name() {
        return this.#__name;
    }
    set name(value: string) {
        this.#__name = value;
    }
}

```

```

    constructor(name: string) {
        this.name = name;
    }
}

```

this

No TypeScript, a palavra-chave `this` refere-se à instância atual de uma classe dentro de seus métodos ou construtores. Ela permite acessar e modificar as propriedades e métodos da classe de dentro de seu próprio escopo. Fornece uma maneira de acessar e manipular o estado interno de um objeto dentro de seus próprios métodos.

```

class Person {
    private name: string;
    constructor(name: string) {
        this.name = name;
    }
    public introduce(): void {
        console.log(`Olá, meu nome é ${this.name}.`);
    }
}

```

```

const person1 = new Person('Alice');
person1.introduce(); // Olá, meu nome é Alice.

```

Propriedades de Parâmetro

As propriedades de parâmetro permitem declarar e inicializar propriedades de classe diretamente dentro dos parâmetros do construtor, evitando o código repetitivo (boilerplate). Exemplo:

```

class Person {
    constructor(
        private name: string,
        public age: number
    ) {
        // As palavras-chave "private" e "public" no construtor
    }
}

```

```

        // declaram e inicializam automaticamente as propriedades
de classe correspondentes.
    }
    public introduce(): void {
        console.log(
            `Olá, meu nome é ${this.name} e eu tenho ${this.age}
anos.`
        );
    }
}
const person = new Person('Alice', 25);
person.introduce();

```

Classes Abstratas

Classes Abstratas são usadas no TypeScript principalmente para herança; elas fornecem uma maneira de definir propriedades e métodos comuns que podem ser herdados por subclasses. Isso é útil quando você deseja definir um comportamento comum e garantir que as subclasses implementem certos métodos. Elas fornecem uma maneira de criar uma hierarquia de classes onde a classe base abstrata fornece uma interface compartilhada e funcionalidade comum para as subclasses.

```

abstract class Animal {
    protected name: string;

    constructor(name: string) {
        this.name = name;
    }

    abstract makeSound(): void;
}

class Cat extends Animal {
    makeSound(): void {
        console.log(`${this.name} mia (meows).`);
    }
}

```

```
const cat = new Cat('Whiskers');  
cat.makeSound(); // Saída: Whiskers mia (meows).
```

Com Genéricos

Classes com genéricos permitem definir classes reutilizáveis que podem trabalhar com diferentes tipos.

```
class Container<T> {  
    private item: T;  
  
    constructor(item: T) {  
        this.item = item;  
    }  
  
    getItem(): T {  
        return this.item;  
    }  
  
    setItem(item: T): void {  
        this.item = item;  
    }  
}  
  
const container1 = new Container<number>(42);  
console.log(container1.getItem()); // 42  
  
const container2 = new Container<string>('Olá');  
container2.setItem('Mundo');  
console.log(container2.getItem()); // Mundo
```

Decoradores (Decorators)

Os decoradores fornecem um mecanismo para adicionar metadados, modificar comportamentos, validar ou estender a funcionalidade do elemento alvo. São funções que são executadas em tempo de execução. Múltiplos decoradores podem ser aplicados a uma declaração.

Os decoradores são recursos experimentais, e os exemplos a seguir são compatíveis apenas com a versão 5 do TypeScript ou superior usando ES6.

Para versões do TypeScript anteriores à 5, eles devem ser habilitados usando a propriedade `experimentalDecorators` em seu `tsconfig.json` ou usando `--experimentalDecorators` na sua linha de comando (mas o exemplo a seguir não funcionará).

Alguns dos casos de uso comuns para decoradores incluem:

- Observar mudanças de propriedade.
- Observar chamadas de métodos.
- Adicionar propriedades ou métodos extras.
- Validação em tempo de execução.
- Serialização e desserialização automática.
- Registro (Logging).
- Autorização e autenticação.
- Proteção contra erros (Error guarding).

Nota: Decoradores para a versão 5 não permitem decorar parâmetros.

Tipos de decoradores:

Decoradores de Classe (Class Decorators)

Os Decoradores de Classe são úteis para estender uma classe existente, como adicionar propriedades ou métodos, ou coletar instâncias de uma classe. No exemplo a seguir, adicionamos um método `toString` que converte a classe em uma representação de string.

```
type Constructor<T = {}> = new (...args: any[]) => T;
```

```
function toString<Class extends Constructor>(  
    Value: Class,
```

```

    context: ClassDecoratorContext<Class>
  ) {
    return class extends Value {
      constructor(...args: any[]) {
        super(...args);
        console.log(JSON.stringify(this));
        console.log(JSON.stringify(context));
      }
    };
  }
}

@toString
class Person {
  name: string;

  constructor(name: string) {
    this.name = name;
  }

  greet() {
    return 'Hello, ' + this.name;
  }
}

const person = new Person('Simon');
/* Registra:
{"name":"Simon"}
{"kind":"class","name":"Person"}
*/

```

Decorador de Propriedade (Property Decorator)

Os decoradores de propriedade são úteis para modificar o comportamento de uma propriedade, como alterar os valores de inicialização. No código a seguir, temos um script que define uma propriedade para estar sempre em letras maiúsculas:

```

function upperCase<T>(
  target: undefined,
  context: ClassFieldDecoratorContext<T, string>
) {
  return function (this: T, value: string) {
    return value.toUpperCase();
  };
}

```

```

    };
}

class MyClass {
    @uppercase
    prop1 = 'hello!';
}

console.log(new MyClass().prop1); // Registra: HELLO!

```

Decorador de Método (Method Decorator)

Os decoradores de método permitem alterar ou aprimorar o comportamento dos métodos. Abaixo está um exemplo de um registrador (logger) simples:

```

function log<This, Args extends any[], Return>(
    target: (this: This, ...args: Args) => Return,
    context: ClassMethodDecoratorContext<
        This,
        (this: This, ...args: Args) => Return
    >
) {
    const methodName = String(context.name);

    function replacementMethod(this: This, ...args: Args): Return {
        console.log(`LOG: Entrando no método '${methodName}'.`);
        const result = target.call(this, ...args);
        console.log(`LOG: Saindo do método '${methodName}'.`);
        return result;
    }

    return replacementMethod;
}

class MyClass {
    @log
    sayHello() {
        console.log('Hello!');
    }
}

```

```
new MyClass().sayHello();
```

Isso registra:

```
LOG: Entrando no método 'sayHello'.  
Hello!  
LOG: Saindo do método 'sayHello'.
```

Decoradores de Getter e Setter

Decoradores de getter e setter permitem alterar ou aprimorar o comportamento dos acessores de classe. Eles são úteis, por exemplo, para validar atribuições de propriedades. Aqui está um exemplo simples de um decorador de getter:

```
function range<This, Return extends number>(min: number, max: number) {  
    return function (  
        target: (this: This) => Return,  
        context: ClassGetterDecoratorContext<This, Return>  
    ) {  
        return function (this: This): Return {  
            const value = target.call(this);  
            if (value < min || value > max) {  
                throw 'Invalid';  
            }  
            Object.defineProperty(this, context.name, {  
                value,  
                enumerable: true,  
            });  
            return value;  
        };  
    };  
}  
  
class MyClass {  
    private _value = 0;  
  
    constructor(value: number) {  
        this._value = value;  
    }  
}
```

```

    @range(1, 100)
    get getValue(): number {
        return this._value;
    }
}

const obj = new MyClass(10);
console.log(obj.getValue); // Válido: 10

const obj2 = new MyClass(999);
console.log(obj2.getValue); // Lança: Invalid!

```

Metadados de Decorador (Decorator Metadata)

Os Metadados de Decorador simplificam o processo para que os decoradores apliquem e utilizem metadados em qualquer classe. Eles podem acessar uma nova propriedade de metadados no objeto de contexto, que pode servir como uma chave para primitivos e objetos. As informações de metadados podem ser acessadas na classe via `Symbol.metadata`.

Os metadados podem ser usados para vários fins, como depuração, serialização ou injeção de dependência com decoradores.

```

//@ts-ignore
Symbol.metadata ??= Symbol('Symbol.metadata'); // Polify simples

type Context =
    | ClassFieldDecoratorContext
    | ClassAccessorDecoratorContext
    | ClassMethodDecoratorContext; // O contexto contém os
metadados da propriedade: DecoratorMetadata

function setMetadata(_target: any, context: Context) {
    // Define o objeto de metadados com um valor primitivo
    context.metadata[context.name] = true;
}

class MyClass {
    @setMetadata
    a = 123;
}

```

```

    @setMetadata
    accessor b = 'b';

    @setMetadata
    fn() {}
}

const metadata = MyClass[Symbol.metadata]; // Obtém as informações
de metadados

console.log(JSON.stringify(metadata)); //
{"bar":true,"baz":true,"foo":true}

```

Herança

Herança refere-se ao mecanismo pelo qual uma classe pode herdar propriedades e métodos de outra classe, conhecida como classe base ou superclasse. A classe derivada, também chamada de classe filha ou subclasse, pode estender e especializar a funcionalidade da classe base adicionando novas propriedades e métodos ou substituindo (overriding) os existentes.

```

class Animal {
    name: string;

    constructor(name: string) {
        this.name = name;
    }

    speak(): void {
        console.log('O animal faz um som');
    }
}

class Dog extends Animal {
    breed: string;

    constructor(name: string, breed: string) {
        super(name);
    }
}

```

```

        this.breed = breed;
    }

    speak(): void {
        console.log('Woof! Woof!');
    }
}

// Cria uma instância da classe base
const animal = new Animal('Animal Genérico');
animal.speak(); // O animal faz um som

// Cria uma instância da classe derivada
const dog = new Dog('Max', 'Labrador');
dog.speak(); // Woof! Woof!"

```

O TypeScript não suporta herança múltipla no sentido tradicional e, em vez disso, permite a herança de uma única classe base. O TypeScript suporta múltiplas interfaces. Uma interface pode definir um contrato para a estrutura de um objeto, e uma classe pode implementar múltiplas interfaces. Isso permite que uma classe herde comportamento e estrutura de múltiplas fontes.

```

interface Flyable {
    fly(): void;
}

interface Swimmable {
    swim(): void;
}

class FlyingFish implements Flyable, Swimmable {
    fly() {
        console.log('Voando...');
    }

    swim() {
        console.log('Nadando...');
    }
}

```

```
const flyingFish = new FlyingFish();
flyingFish.fly();
flyingFish.swim();
```

A palavra-chave `class` no TypeScript, assim como no JavaScript, é frequentemente chamada de açúcar sintático (syntactic sugar). Ela foi introduzida no ECMAScript 2015 (ES6) para oferecer uma sintaxe mais familiar para criar e trabalhar com objetos de maneira baseada em classes. No entanto, é importante notar que o TypeScript, sendo um superconjunto do JavaScript, acaba sendo compilado para JavaScript, que permanece baseado em protótipos em seu núcleo.

Estáticos (Statics)

O TypeScript possui membros estáticos. Para acessar os membros estáticos de uma classe, você pode usar o nome da classe seguido por um ponto, sem a necessidade de criar um objeto.

```
class OfficeWorker {
    static memberCount: number = 0;

    constructor(private name: string) {
        OfficeWorker.memberCount++;
    }
}

const w1 = new OfficeWorker('James');
const w2 = new OfficeWorker('Simon');
const total = OfficeWorker.memberCount;
console.log(total); // 2
```

Inicialização de Propriedade

Existem várias maneiras de inicializar propriedades de uma classe no TypeScript:

Em linha (Inline):

No exemplo a seguir, esses valores iniciais serão usados quando uma instância da classe for criada.

```
class MyClass {  
    property1: string = 'valor padrão';  
    property2: number = 42;  
}
```

No construtor:

```
class MyClass {  
    property1: string;  
    property2: number;  
  
    constructor() {  
        this.property1 = 'valor padrão';  
        this.property2 = 42;  
    }  
}
```

Usando parâmetros do construtor:

```
class MyClass {  
    constructor(  
        private property1: string = 'valor padrão',  
        public property2: number = 42  
    ) {  
        // Não há necessidade de atribuir os valores às  
        // propriedades explicitamente.  
    }  
    log() {  
        console.log(this.property2);  
    }  
}  
  
const x = new MyClass();  
x.log();
```

Sobrecarga de Método

A sobrecarga de método permite que uma classe tenha múltiplos métodos com o mesmo nome, mas diferentes tipos de parâmetros ou

um número diferente de parâmetros. Isso nos permite chamar um método de diferentes maneiras com base nos argumentos passados.

```
class MyClass {
  add(a: number, b: number): number; // Assinatura de sobrecarga
1
  add(a: string, b: string): string; // Assinatura de sobrecarga
2

  add(a: number | string, b: number | string): number | string {
    if (typeof a === 'number' && typeof b === 'number') {
      return a + b;
    }
    if (typeof a === 'string' && typeof b === 'string') {
      return a.concat(b);
    }
    throw new Error('Argumentos inválidos');
  }
}

const r = new MyClass();
console.log(r.add(10, 5)); // Registra 15
```

Genéricos (Generics)

Os genéricos permitem que você crie componentes e funções reutilizáveis que podem trabalhar com múltiplos tipos. Com os genéricos, você pode parametrizar tipos, funções e interfaces, permitindo que operem em diferentes tipos sem especificá-los explicitamente de antemão.

Os genéricos permitem tornar o código mais flexível e reutilizável.

Tipo Genérico

Para definir um tipo genérico, você usa colchetes angulares (<>) para especificar os parâmetros de tipo, por exemplo:

```

function identity<T>(arg: T): T {
    return arg;
}
const a = identity('x');
const b = identity(123);

const getLen = <T,>(data: ReadonlyArray<T>) => data.length;
const len = getLen([1, 2, 3]);

```

Classes Genéricas

Os genéricos também podem ser aplicados a classes, permitindo que trabalhem com múltiplos tipos por meio de parâmetros de tipo. Isso é útil para criar definições de classe reutilizáveis que podem operar em diferentes tipos de dados mantendo a segurança de tipo.

```

class Container<T> {
    private item: T;

    constructor(item: T) {
        this.item = item;
    }

    getItem(): T {
        return this.item;
    }
}

const numberContainer = new Container<number>(123);
console.log(numberContainer.getItem()); // 123

const stringContainer = new Container<string>('hello');
console.log(stringContainer.getItem()); // hello

```

Restrições Genéricas (Generic Constraints)

Parâmetros genéricos podem ser restringidos usando a palavra-chave `extends` seguida por um tipo ou interface que o parâmetro de tipo deve satisfazer.

No exemplo a seguir, `T` deve conter uma propriedade `length` para ser válido:

```
const printLen = <T extends { length: number }>(value: T): void =>
{
    console.log(value.length);
};

printLen('Hello'); // 5
printLen([1, 2, 3]); // 3
printLen({ length: 10 }); // 10
printLen(123); // Inválido
```

Um recurso interessante de genéricos introduzido na versão 3.4 RC é a inferência de tipo de função de ordem superior, que introduziu argumentos de tipo genérico propagados:

```
declare function pipe<A extends any[], B, C>(
    ab: (...args: A) => B,
    bc: (b: B) => C
): (...args: A) => C;

declare function list<T>(a: T): T[];
declare function box<V>(x: V): { value: V };

const listBox = pipe(list, box); // <T>(a: T) => { value: T[] }
const boxList = pipe(box, list); // <V>(x: V) => { value: V }[]
```

Essa funcionalidade permite uma programação de estilo sem pontos (pointfree) com segurança de tipo mais fácil, o que é comum na programação funcional.

Estreitamento Contextual Genérico

O estreitamento contextual (contextual narrowing) para genéricos é o mecanismo no TypeScript que permite ao compilador estreitar o tipo de um parâmetro genérico com base no contexto em que é usado. É útil ao trabalhar com tipos genéricos em declarações condicionais:

```
function process<T>(value: T): void {
    if (typeof value === 'string') {
        // O valor é estreitado para o tipo 'string'
        console.log(value.length);
    } else if (typeof value === 'number') {
        // O valor é estreitado para o tipo 'number'
        console.log(value.toFixed(2));
    }
}

process('hello'); // 5
process(3.14159); // 3.14
```

Tipos Estruturais Apagados

No TypeScript, os objetos não precisam corresponder a um tipo exato e específico. Por exemplo, se criarmos um objeto que satisfaça os requisitos de uma interface, podemos utilizar esse objeto em locais onde essa interface é necessária, mesmo que não haja uma conexão explícita entre eles. Exemplo:

```
type NameProp1 = {
    prop1: string;
};

function log(x: NameProp1) {
    console.log(x.prop1);
}

const obj = {
```

```
    prop2: 123,  
    prop1: 'Origin',  
};
```

```
log(obj); // Válido
```

Namespacing

No TypeScript, os namespaces são usados para organizar o código em contêineres lógicos, evitando colisões de nomes e fornecendo uma maneira de agrupar códigos relacionados. O uso das palavras-chave `export` permite o acesso ao namespace em módulos externos.

```
export namespace MyNamespace {  
    export interface MyInterface1 {  
        prop1: boolean;  
    }  
    export interface MyInterface2 {  
        prop2: string;  
    }  
}  
  
const a: MyNamespace.MyInterface1 = {  
    prop1: true,  
};
```

Símbolos (Symbols)

Símbolos são um tipo de dado primitivo que representa um valor imutável que é garantido ser globalmente único durante todo o tempo de execução do programa.

Símbolos podem ser usados como chaves para propriedades de objetos e fornecem uma maneira de criar propriedades não enumeráveis.

```
const key1: symbol = Symbol('key1');
const key2: symbol = Symbol('key2');

const obj = {
  [key1]: 'value 1',
  [key2]: 'value 2',
};

console.log(obj[key1]); // value 1
console.log(obj[key2]); // value 2
```

Em WeakMaps e WeakSets, símbolos agora são permitidos como chaves.

Diretivas Triple-Slash

As diretivas triple-slash são comentários especiais que fornecem instruções ao compilador sobre como processar um arquivo. Essas diretivas começam com três barras consecutivas (///) e são normalmente colocadas no topo de um arquivo TypeScript e não têm efeitos no comportamento em tempo de execução.

As diretivas triple-slash são usadas para referenciar dependências externas, especificar o comportamento de carregamento de módulos, habilitar/desabilitar certos recursos do compilador e muito mais. Alguns exemplos:

Referenciando um arquivo de declaração:

```
/// <reference path="caminho/para/arquivo/de/declaracao.d.ts" />
```

Indicar o formato do módulo:

```
/// <amd|commonjs|system|umd|es6|es2015|none>
```

Habilitar opções do compilador, no exemplo a seguir, o modo estrito:

```
/// <strict|noImplicitAny|noUnusedLocals|noUnusedParameters>
```

Manipulação de Tipos

Criando Tipos a partir de Tipos

É possível criar novos tipos compondo, manipulando ou transformando tipos existentes.

Tipos Interseção (&):

Permitem combinar múltiplos tipos em um único tipo:

```
type A = { foo: number };
type B = { bar: string };
type C = A & B; // Interseção de A e B
const obj: C = { foo: 42, bar: 'hello' };
```

Tipos União (|):

Permitem definir um tipo que pode ser um de vários tipos:

```
type Result = string | number;
const value1: Result = 'hello';
const value2: Result = 42;
```

Tipos Mapeados:

Permitem transformar as propriedades de um tipo existente para criar um novo tipo:

```
type Mutable<T> = {
  readonly [P in keyof T]: T[P];
};
type Person = {
  name: string;
  age: number;
};
type ImmutablePerson = Mutable<Person>; // As propriedades tornam-se somente leitura
```

Tipos Condicionais:

Permitem criar tipos com base em algumas condições:

```
type ExtractParam<T> = T extends (param: infer P) => any ? P :  
never;  
type MyFunction = (name: string) => number;  
type ParamType = ExtractParam<MyFunction>; // string
```

Tipos de Acesso Indexado (Indexed Access Types)

No TypeScript, é possível acessar e manipular os tipos de propriedades dentro de outro tipo usando um índice, `Type[Key]`.

```
type Person = {  
  name: string;  
  age: number;  
};  
  
type AgeType = Person['age']; // number  
  
type MyTuple = [string, number, boolean];  
type MyType = MyTuple[2]; // boolean
```

Tipos Utilitários (Utility Types)

Vários tipos utilitários integrados podem ser usados para manipular tipos, abaixo uma lista dos mais comuns:

Awaited<T>

Constrói um tipo que descompacta recursivamente tipos `Promise`.

```
type A = Awaited<Promise<string>>; // string
```

Partial<T>

Constrói um tipo com todas as propriedades de T definidas como opcionais.

```
type Person = {  
  name: string;  
  age: number;  
};  
  
type A = Partial<Person>; // { name?: string | undefined; age?:  
number | undefined; }
```

Required<T>

Constrói um tipo com todas as propriedades de T definidas como obrigatórias.

```
type Person = {  
  name?: string;  
  age?: number;  
};  
  
type A = Required<Person>; // { name: string; age: number; }
```

Readonly<T>

Constrói um tipo com todas as propriedades de T definidas como somente leitura.

```
type Person = {  
  name: string;  
  age: number;  
};  
  
type A = Readonly<Person>;  
  
const a: A = { name: 'Simon', age: 17 };  
a.name = 'John'; // Inválido
```

Record<K, T>

Constrói um tipo com um conjunto de propriedades K do tipo T.

```
type Product = {  
  name: string;  
  price: number;  
};  
  
const products: Record<string, Product> = {  
  apple: { name: 'Apple', price: 0.5 },  
  banana: { name: 'Banana', price: 0.25 },  
};  
  
console.log(products.apple); // { name: 'Apple', price: 0.5 }
```

Pick<T, K>

Constrói um tipo selecionando as propriedades especificadas K de T.

```
type Product = {  
  name: string;  
  price: number;  
};  
  
type Price = Pick<Product, 'price'>; // { price: number; }
```

Omit<T, K>

Constrói um tipo omitindo as propriedades especificadas K de T.

```
type Product = {  
  name: string;  
  price: number;  
};  
  
type Name = Omit<Product, 'price'>; // { name: string; }
```

Exclude<T, U>

Constrói um tipo excluindo todos os valores do tipo U de T.

```
type Union = 'a' | 'b' | 'c';
type MyType = Exclude<Union, 'a' | 'c'>; // b
```

Extract<T, U>

Constrói um tipo extraindo todos os valores do tipo U de T.

```
type Union = 'a' | 'b' | 'c';
type MyType = Extract<Union, 'a' | 'c'>; // a | c
```

NonNullable<T>

Constrói um tipo excluindo null e undefined de T.

```
type Union = 'a' | null | undefined | 'b';
type MyType = NonNullable<Union>; // 'a' | 'b'
```

Parameters<T>

Extrai os tipos de parâmetros de um tipo de função T.

```
type Func = (a: string, b: number) => void;
type MyType = Parameters<Func>; // [a: string, b: number]
```

ConstructorParameters<T>

Extrai os tipos de parâmetros de um tipo de função construtora T.

```
class Person {
  constructor(
    public name: string,
    public age: number
  ) {}
}
```

```

type PersonConstructorParams = ConstructorParameters<typeof
Person>; // [name: string, age: number]
const params: PersonConstructorParams = ['John', 30];
const person = new Person(...params);
console.log(person); // Person { name: 'John', age: 30 }

```

ReturnType<T>

Extraí o tipo de retorno de um tipo de função T.

```

type Func = (name: string) => number;
type MyType = ReturnType<Func>; // number

```

InstanceType<T>

Extraí o tipo de instância de um tipo de classe T.

```

class Person {
  name: string;

  constructor(name: string) {
    this.name = name;
  }

  sayHello() {
    console.log(`Olá, meu nome é ${this.name}!`);
  }
}

```

```

type PersonInstance = InstanceType<typeof Person>;

```

```

const person: PersonInstance = new Person('John');

```

```

person.sayHello(); // Olá, meu nome é John!

```

ThisParameterType<T>

Extraí o tipo do parâmetro 'this' de um tipo de função T.

```
interface Person {
    name: string;
    greet(this: Person): void;
}
type PersonThisType = ThisParameterType<Person['greet']>; // Person
```

OmitThisParameter<T>

Remove o parâmetro 'this' de um tipo de função T.

```
function capitalize(this: String) {
    return this[0].toUpperCase() + this.substring(1).toLowerCase();
}

type CapitalizeType = OmitThisParameter<typeof capitalize>; // ()
=> string
```

ThisType<T>

Serve como um marcador para um tipo this contextual.

```
type Logger = {
    log: (error: string) => void;
};

let helperFunctions: { [name: string]: Function } &
ThisType<Logger> = {
    hello: function () {
        this.log('some error'); // Válido, pois "log" faz parte de
        "this".
        this.update(); // Inválido
    },
};
```

Uppercase<T>

Converte para maiúsculas o nome do tipo de entrada T.

```
type MyType = Uppercase<'abc'>; // "ABC"
```

Lowercase<T>

Converte para minúsculas o nome do tipo de entrada T.

```
type MyType = Lowercase<'ABC'>; // "abc"
```

Capitalize<T>

Coloca a primeira letra em maiúscula no nome do tipo de entrada T.

```
type MyType = Capitalize<'abc'>; // "Abc"
```

Uncapitalize<T>

Coloca a primeira letra em minúscula no nome do tipo de entrada T.

```
type MyType = Uncapitalize<'Abc'>; // "abc"
```

NoInfer<T>

NoInfer é um tipo utilitário projetado para bloquear a inferência automática de tipos dentro do escopo de uma função genérica.

Exemplo:

```
// Inferência automática de tipos dentro do escopo de uma função genérica.  
function fn<T extends string>(x: T[], y: T) {  
    return x.concat(y);  
}  
const r = fn(['a', 'b'], 'c'); // O tipo aqui é ("a" | "b" | "c")[]
```

Com NoInfer:

```
// Exemplo de função que usa NoInfer para evitar inferência de tipo  
function fn2<T extends string>(x: T[], y: NoInfer<T>) {  
    return x.concat(y);  
}
```

```
const r2 = fn2(['a', 'b'], 'c'); // Erro: Argumento de tipo '"c"'  
não é atribuível ao parâmetro do tipo '"a" | "b"'.  

```

Outros

Erros e Tratamento de Exceções

O TypeScript permite capturar e tratar erros usando os mecanismos padrão de tratamento de erros do JavaScript:

Blocos Try-Catch-Finally:

```
try {  
    // Código que pode lançar um erro  
} catch (error) {  
    // Trata o erro  
} finally {  
    // Código que sempre executa, finally é opcional  
}
```

Você também pode tratar diferentes tipos de erro:

```
try {  
    // Código que pode lançar diferentes tipos de erros  
} catch (error) {  
    if (error instanceof TypeError) {  
        // Trata TypeError  
    } else if (error instanceof RangeError) {  
        // Trata RangeError  
    } else {  
        // Trata outros erros  
    }  
}
```

Tipos de Erro Personalizados:

É possível especificar erros mais específicos estendendo a class Error:

```
class CustomError extends Error {
  constructor(message: string) {
    super(message);
    this.name = 'CustomError';
  }
}

throw new CustomError('Este é um erro personalizado.');
```

Classes Mixin (Mixin classes)

As classes mixin permitem combinar e compor comportamentos de múltiplas classes em uma única classe. Elas fornecem uma maneira de reutilizar e estender funcionalidades sem a necessidade de cadeias de herança profundas.

```
abstract class Identifiable {
  name: string = '';
  logId() {
    console.log('id:', this.name);
  }
}

abstract class Selectable {
  selected: boolean = false;
  select() {
    this.selected = true;
    console.log('Select');
  }
  deselect() {
    this.selected = false;
    console.log('Deselect');
  }
}

class MyClass {
  constructor() {}
}
```

```

// Estende MyClass para incluir o comportamento de Identifiable e
Selectable
interface MyClass extends Identifiable, Selectable {}

// Função para aplicar mixins a uma classe
function applyMixins(source: any, baseCtors: any[]) {
  baseCtors.forEach(baseCtor => {
    Object.getOwnPropertyNames(baseCtor.prototype).forEach(name
=> {
      let descriptor = Object.getOwnPropertyDescriptor(
        baseCtor.prototype,
        name
      );
      if (descriptor) {
        Object.defineProperty(source.prototype, name,
descriptor);
      }
    });
  });
}

// Aplica os mixins a MyClass
applyMixins(MyClass, [Identifiable, Selectable]);
let o = new MyClass();
o.name = 'abc';
o.logId();
o.select();

```

Recursos de Linguagem Assíncronos

Como o TypeScript é um superconjunto do JavaScript, ele possui recursos de linguagem assíncronos integrados do JavaScript como:

Promises:

Promises são uma maneira de lidar com operações assíncronas e seus resultados usando métodos como `.then()` e `.catch()` para lidar com condições de sucesso e erro.

Para saber mais: https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Global_Objects/Promise

Async/await:

As palavras-chave `async/await` são uma maneira de fornecer uma sintaxe com aparência mais síncrona para trabalhar com Promises. A palavra-chave `async` é usada para definir uma função assíncrona, e a palavra-chave `await` é usada dentro de uma função `async` para pausar a execução até que uma Promise seja resolvida ou rejeitada.

Para saber mais: https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Statements/async_function
<https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Operators/await>

As seguintes APIs são bem suportadas no TypeScript:

Fetch API: https://developer.mozilla.org/pt-BR/docs/Web/API/Fetch_API

Web Workers: https://developer.mozilla.org/pt-BR/docs/Web/API/Web_Workers_API

Shared Workers: <https://developer.mozilla.org/pt-BR/docs/Web/API/SharedWorker>

WebSocket: https://developer.mozilla.org/pt-BR/docs/Web/API/WebSockets_API

Iteradores e Geradores

Tanto Iteradores quanto Geradores são bem suportados no TypeScript.

Iteradores são objetos que implementam o protocolo iterador, fornecendo uma maneira de acessar elementos de uma coleção ou sequência um por um. É uma estrutura que contém um ponteiro para o próximo elemento na iteração. Eles possuem um método `next()` que retorna o próximo valor na sequência junto com um booleano indicando se a sequência terminou (`done`).

```
class NumberIterator implements Iterable<number> {
  private current: number;

  constructor(
    private start: number,
    private end: number
  ) {
    this.current = start;
  }

  public next(): IteratorResult<number> {
    if (this.current <= this.end) {
      const value = this.current;
      this.current++;
      return { value, done: false };
    } else {
      return { value: undefined, done: true };
    }
  }

  [Symbol.iterator]() {
    return this;
  }
}

const iterator = new NumberIterator(1, 3);

for (const num of iterator) {
  console.log(num);
}
```

Geradores são funções especiais definidas usando a sintaxe `function*` que simplifica a criação de iteradores. Eles usam a palavra-chave

`yield` para definir a sequência de valores e pausar e retomar automaticamente a execução quando os valores são solicitados.

Geradores facilitam a criação de iteradores e são especialmente úteis para trabalhar com sequências grandes ou infinitas.

Exemplo:

```
function* numberGenerator(start: number, end: number):
Generator<number> {
    for (let i = start; i <= end; i++) {
        yield i;
    }
}

const generator = numberGenerator(1, 5);

for (const num of generator) {
    console.log(num);
}
```

O TypeScript também suporta iteradores assíncronos e geradores assíncronos.

Para saber mais:

https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Global_Objects/Generator

https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Global_Objects/Iterator

Referência JSDoc TsDocs

Ao trabalhar com uma base de código JavaScript, é possível ajudar o TypeScript a inferir o tipo correto usando comentários JSDoc com anotações adicionais para fornecer informações de tipo.

Exemplo:

```
/**
 * Computa a potência de um dado número
 * @constructor
 * @param {number} base – O valor da base da expressão
 * @param {number} exponent – O valor do expoente da expressão
 */
function power(base: number, exponent: number) {
    return Math.pow(base, exponent);
}
power(10, 2); // function power(base: number, exponent: number):
              number
```

A documentação completa é fornecida neste link:

<https://www.typescriptlang.org/docs/handbook/jsdoc-supported-types.html>

A partir da versão 3.7, é possível gerar definições de tipo .d.ts a partir da sintaxe JSDoc do JavaScript. Mais informações podem ser encontradas aqui:

<https://www.typescriptlang.org/docs/handbook/declaration-files/dts-from-js.html>

@types

Pacotes sob a organização @types são convenções especiais de nomenclatura de pacotes usadas para fornecer definições de tipo para bibliotecas ou módulos JavaScript existentes. Por exemplo, usando:

```
npm install --save-dev @types/lodash
```

Instalará as definições de tipo de lodash em seu projeto atual.

Para contribuir com as definições de tipo do pacote @types, envie um pull request para

<https://github.com/DefinitelyTyped/DefinitelyTyped>.

JSX

JSX (JavaScript XML) é uma extensão da sintaxe da linguagem JavaScript que permite escrever código semelhante a HTML em seus arquivos JavaScript ou TypeScript. É comumente usado no React para definir a estrutura HTML.

O TypeScript estende as capacidades do JSX fornecendo verificação de tipos e análise estática.

Para usar JSX, você precisa definir a opção de compilador `jsx` em seu arquivo `tsconfig.json`. Duas opções de configuração comuns:

- “preserve”: emite arquivos `.jsx` com o JSX inalterado. Esta opção diz ao TypeScript para manter a sintaxe JSX como está e não transformá-la durante o processo de compilação. Você pode usar esta opção se tiver uma ferramenta separada, como o Babel, que lida com a transformação.
- “react”: habilita a transformação JSX integrada do TypeScript. `React.createElement` será usado.

Todas as opções estão disponíveis aqui:

<https://www.typescriptlang.org/tsconfig#jsx>

Módulos ES6

O TypeScript suporta ES6 (ECMAScript 2015) e muitas versões subsequentes. Isso significa que você pode usar a sintaxe ES6, como arrow functions, template literals, classes, módulos, desestruturação e muito mais.

Para habilitar recursos do ES6 em seu projeto, você pode especificar a propriedade `target` no `tsconfig.json`.

Um exemplo de configuração:

```
{
  "compilerOptions": {
    "target": "es6",
    "module": "es6",
    "moduleResolution": "node",
    "sourceMap": true,
    "outDir": "dist"
  },
  "include": ["src"]
}
```

Operador de Exponenciação ES7

O operador de exponenciação (**) calcula o valor obtido elevando o primeiro operando à potência do segundo operando. Ele funciona de forma semelhante a `Math.pow()`, mas com a capacidade adicional de aceitar BigInts como operandos. O TypeScript suporta totalmente este operador usando como `target` no seu arquivo `tsconfig.json` a versão `es2016` ou superior.

```
console.log(2 ** (2 ** 2)); // 16
```

A Instrução `for-await-of`

Este é um recurso do JavaScript totalmente suportado no TypeScript que permite iterar sobre objetos iteráveis assíncronos a partir da versão de destino `es2018`.

```
async function* asyncNumbers(): AsyncIterableIterator<number> {
  yield Promise.resolve(1);
  yield Promise.resolve(2);
  yield Promise.resolve(3);
}

(async () => {
  for await (const num of asyncNumbers()) {
    console.log(num);
  }
})();
```

Nova meta-propriedade target

Você pode usar no TypeScript a meta-propriedade `new.target` que permite determinar se uma função ou construtor foi invocado usando o operador `new`. Ela permite detectar se um objeto foi criado como resultado de uma chamada de construtor.

```
class Parent {
    constructor() {
        console.log(new.target); // Registra a função construtora
        // usada para criar uma instância
    }
}

class Child extends Parent {
    constructor() {
        super();
    }
}

const parentX = new Parent(); // [Function: Parent]
const child = new Child(); // [Function: Child]
```

Expressões de Importação Dinâmica

É possível carregar módulos condicionalmente ou carregá-los sob demanda (lazy load) usando a proposta do ECMAScript para importação dinâmica, que é suportada no TypeScript.

A sintaxe para expressões de importação dinâmica no TypeScript é as seguinte:

```
async function renderWidget() {
    const container = document.getElementById('widget');
    if (container !== null) {
        const widget = await import('./widget'); // Importação
        // dinâmica
        widget.render(container);
    }
}
```

```
}
```

```
renderWidget();
```

“tsc --watch”

Este comando inicia o compilador TypeScript com o parâmetro `--watch`, com a capacidade de recompilar automaticamente os arquivos TypeScript sempre que forem modificados.

```
tsc --watch
```

A partir da versão 4.9 do TypeScript, o monitoramento de arquivos depende principalmente de eventos do sistema de arquivos, recorrendo automaticamente à sondagem (polling) se um observador baseado em eventos não puder ser estabelecido.

Operador de Asserção Não-Nulo (Non-null Assertion Operator)

O Operador de Asserção Não-Nulo (Postfix `!`) também chamado de Asserções de Atribuição Definitiva (Definite Assignment Assertions) é um recurso do TypeScript que permite asseverar que uma variável ou propriedade não é nula ou indefinida, mesmo que a análise de tipo estática do TypeScript sugira que poderia ser. Com este recurso, é possível remover qualquer verificação explícita.

```
type Person = {  
    name: string;  
};
```

```
const printName = (person?: Person) => {  
    console.log(`O nome é ${person!.name}`);  
};
```

Declarações com Valor Padrão (Defaulted declarations)

Declarações com valor padrão são usadas quando uma variável ou parâmetro recebe um valor padrão. Isso significa que se nenhum valor for fornecido para essa variável ou parâmetro, o valor padrão será usado no lugar.

```
function greet(name: string = 'Anônimo'): void {  
    console.log(`Olá, ${name}!`);  
}  
greet(); // Olá, Anônimo!  
greet('John'); // Olá, John!
```

Encadeamento Opcional (Optional Chaining)

O operador de encadeamento opcional `?.` funciona como o operador de ponto regular `.` para acessar propriedades ou métodos. No entanto, ele trata graciosamente valores nulos ou indefinidos terminando a expressão e retornando `undefined`, em vez de lançar um erro.

```
type Person = {  
    name: string;  
    age?: number;  
    address?: {  
        street?: string;  
        city?: string;  
    };  
};  
  
const person: Person = {  
    name: 'John',  
};  
  
console.log(person.address?.city); // undefined
```

Operador de Coalescência Nula (Nullish coalescing operator)

O operador de coalescência nula ?? retorna o valor do lado direito se o lado esquerdo for `null` ou `undefined`; caso contrário, retorna o valor do lado esquerdo.

```
const foo = null ?? 'foo';  
console.log(foo); // foo
```

```
const baz = 1 ?? 'baz';  
const baz2 = 0 ?? 'baz';  
console.log(baz); // 1  
console.log(baz2); // 0
```

Tipos de Literal de Template (Template Literal Types)

Tipos de Literal de Template permitem manipular valores de string em nível de tipo e gerar novos tipos de string baseados em existentes. Eles são úteis para criar tipos mais expressivos e precisos a partir de operações baseadas em string.

```
type Department = 'engineering' | 'hr';  
type Language = 'english' | 'spanish';  
type Id = `${Department}-${Language}-id`; // "engineering-english-id" | "engineering-spanish-id" | "hr-english-id" | "hr-spanish-id"
```

Sobrecarga de Função (Function overloading)

A sobrecarga de função permite definir múltiplas assinaturas de função para o mesmo nome de função, cada uma com diferentes tipos de parâmetros e tipo de retorno. Quando você chama uma

função sobrecarregada, o TypeScript usa os argumentos fornecidos para determinar a assinatura de função correta:

```
function makeGreeting(name: string): string;
function makeGreeting(names: string[]): string[];

function makeGreeting(person: unknown): unknown {
    if (typeof person === 'string') {
        return `Hi ${person}!`;
    } else if (Array.isArray(person)) {
        return person.map(name => `Hi, ${name}!`);
    }
    throw new Error('Não foi possível saudar');
}

makeGreeting('Simon');
makeGreeting(['Simone', 'John']);
```

Tipos Recursivos

Um Tipo Recursivo é um tipo que pode se referir a si mesmo. Isso é útil para definir estruturas de dados que possuem uma estrutura hierárquica ou recursiva (aninhamento potencialmente infinito), como listas ligadas, árvores e grafos.

```
type ListNode<T> = {
    data: T;
    next: ListNode<T> | undefined;
};
```

Tipos Condicionais Recursivos

É possível definir relacionamentos de tipo complexos usando lógica e recursão no TypeScript. Vamos detalhar em termos simples:

Tipos Condicionais: permite definir tipos baseados em condições booleanas:

```

type CheckNumber<T> = T extends number ? 'Número' : 'Não é um
número';
type A = CheckNumber<123>; // 'Número'
type B = CheckNumber<'abc'>; // 'Não é um número'

```

Recursão: significa uma definição de tipo que se refere a si mesma dentro de sua própria definição:

```

type Json = string | number | boolean | null | Json[] | { [key:
string]: Json };

```

```

const data: Json = {
  prop1: true,
  prop2: 'prop2',
  prop3: {
    prop4: [],
  },
};

```

Tipos Condicionais Recursivos combinam lógica condicional e recursão. Isso significa que uma definição de tipo pode depender de si mesma através de lógica condicional, criando relacionamentos de tipo complexos e flexíveis.

```

type Flatten<T> = T extends Array<infer U> ? Flatten<U> : T;

type NestedArray = [1, [2, [3, 4], 5], 6];
type FlattenedArray = Flatten<NestedArray>; // 2 | 3 | 4 | 5 | 1 |
6

```

Suporte a Módulos ECMAScript no Node

O Node.js adicionou suporte para Módulos ECMAScript a partir da versão 15.3.0, e o TypeScript tem suporte a Módulos ECMAScript para Node.js desde a versão 4.7. Este suporte pode ser habilitado usando a propriedade `module` com o valor `nodenext` no arquivo `tsconfig.json`. Aqui está um exemplo:

```
{
  "compilerOptions": {
    "module": "nodenext",
    "outDir": "./lib",
    "declaration": true
  }
}
```

O Node.js suporta duas extensões de arquivo para módulos: `.mjs` para módulos ES e `.cjs` para módulos CommonJS. As extensões de arquivo equivalentes no TypeScript são `.mts` para módulos ES e `.cts` para módulos CommonJS. Quando o compilador TypeScript transpila esses arquivos para JavaScript, ele criará arquivos `.mjs` e `.cjs`.

Se você quiser usar módulos ES em seu projeto, você pode definir a propriedade `type` como “module” em seu arquivo `package.json`. Isso instrui o Node.js a tratar o projeto como um projeto de módulo ES.

Além disso, o TypeScript também suporta declarações de tipo em arquivos `.d.ts`. Esses arquivos de declaração fornecem informações de tipo para bibliotecas ou módulos escritos em TypeScript, permitindo que outros desenvolvedores os utilizem com a verificação de tipo e os recursos de preenchimento automático do TypeScript.

Funções de Asserção (Assertion Functions)

No TypeScript, funções de asserção são funções que indicam a verificação de uma condição específica com base em seu valor de retorno. Em sua forma mais simples, uma função `assert` examina um predicado fornecido e lança um erro quando o predicado é avaliado como falso.

```
function isNumber(value: unknown): asserts value is number {
  if (typeof value !== 'number') {
    throw new Error('Não é um número');
  }
}
```

```
}  
}
```

Ou pode ser declarada como uma expressão de função:

```
type AssertIsNumber = (value: unknown) => asserts value is number;  
const isNumber: AssertIsNumber = value => {  
    if (typeof value !== 'number') {  
        throw new Error('Não é um número');  
    }  
};
```

Funções de asserção compartilham semelhanças com guardas de tipo (type guards). As guardas de tipo foram inicialmente introduzidas para realizar verificações em tempo de execução e garantir o tipo de um valor dentro de um escopo específico. Especificamente, uma guarda de tipo é uma função que avalia um predicado de tipo e retorna um valor booleano indicando se the predicado é verdadeiro ou falso. Isso difere ligeiramente das funções de asserção, onde a intenção é lançar um erro em vez de retornar falso quando o predicado não for satisfeito.

Exemplo de guarda de tipo:

```
const isNumber = (value: unknown): value is number => typeof value  
=== 'number';
```

Tipos de Tupla Variádicos (Variadic Tuple Types)

Tipos de Tupla Variádicos são recursos introduzidos na versão 4.0 do TypeScript. Vamos começar revisando o que é uma tupla:

Um tipo tupla é um array que possui um comprimento definido, e onde o tipo de cada elemento é conhecido:

```
type Student = [string, number];  
const [name, age]: Student = ['Simone', 20];
```

O termo “variádico” significa aridade indefinida (aceita um número variável de argumentos).

Uma tupla variádica é um tipo tupla que possui todas as propriedades anteriores, mas o formato exato ainda não está definido:

```
type Bar<T extends unknown[]> = [boolean, ...T, number];
```

```
type A = Bar<[boolean]>; // [boolean, boolean, number]
type B = Bar<['a', 'b']>; // [boolean, 'a', 'b', number]
type C = Bar<[]>; // [boolean, number]
```

No código anterior, podemos ver que o formato da tupla é definido pelo genérico `T` passado.

Tuplas variádicas podem aceitar múltiplos genéricos, tornando-as muito flexíveis:

```
type Bar<T extends unknown[], G extends unknown[]> = [...T,
boolean, ...G];

type A = Bar<[number], [string]>; // [number, boolean, string]
type B = Bar<['a', 'b'], [boolean]>; // ["a", "b", boolean,
boolean]
```

Com as novas tuplas variádicas podemos usar:

- Os espalhamentos (spreads) na sintaxe de tipo tupla agora podem ser genéricos, assim podemos representar operações de ordem superior em tuplas e arrays mesmo quando não conhecemos os tipos reais sobre os quais estamos operando.
- Os elementos de resto (rest elements) podem ocorrer em qualquer lugar em uma tupla.

Exemplo:

```
type Items = readonly unknown[];
```

```
function concat<T extends Items, U extends Items>(
  arr1: T,
  arr2: U
): [...T, ...U] {
  return [...arr1, ...arr2];
}

concat([1, 2, 3], ['4', '5', '6']); // [1, 2, 3, "4", "5", "6"]
```

Tipos Boxed (Boxed types)

Tipos boxed referem-se aos objetos de empacotamento que são usados para representar tipos primitivos como objetos. Esses objetos de empacotamento fornecem funcionalidades e métodos adicionais que não estão disponíveis diretamente nos valores primitivos.

Quando você acessa um método como `charAt` ou `normalize` em um primitivo `string`, o JavaScript o empacota em um objeto `String`, chama o método e depois descarta o objeto.

Demonstração:

```
const originalNormalize = String.prototype.normalize;
String.prototype.normalize = function () {
  console.log(this, typeof this);
  return originalNormalize.call(this);
};
console.log('\u0041'.normalize());
```

O TypeScript representa essa diferenciação fornecendo tipos separados para os primitivos e seus empacotadores de objeto correspondentes:

- `string => String`
- `number => Number`
- `boolean => Boolean`
- `symbol => Symbol`
- `bigint => BigInt`

Os tipos boxed geralmente não são necessários. Evite usar tipos boxed e, em vez disso, use o tipo para os primitivos, por exemplo, `string` em vez de `String`.

Covariância e Contravariância no TypeScript

Covariância e contravariância descrevem como as relações de tipo se comportam em tipos genéricos.

Em TypeScript:

- Arrays são **covariantes**, mas isso não é totalmente seguro em termos de tipos.
- Os tipos de parâmetros de funções são:
 - **contravariantes** quando `strictFunctionTypes` está habilitado
 - **bivariantes** caso contrário

Covariância significa que a relação é preservada: se o tipo A é um subtipo de B, então $F<A>$ também é um subtipo de F. Em TypeScript, isso aparece comumente em tipos de retorno e em arrays (embora a covariância de arrays não seja totalmente type-safe).

Contravariância significa que a relação é invertida: se o tipo A é um subtipo de B, então F é um subtipo de $F<A>$. Em TypeScript, os tipos de parâmetros de funções são projetados para serem contravariantes, o que significa que uma função que aceita um tipo mais amplo pode ser usada onde um tipo mais restrito é esperado.

No entanto, na prática, o TypeScript frequentemente permite bivariância para parâmetros de função (a menos que `strictFunctionTypes` esteja habilitado), o que significa que ambas as direções podem ser aceitas mesmo quando isso não é estritamente seguro em termos de tipos.

Exemplo: Imagine um espaço para todos os animais e um espaço separado apenas para cães.

- **Covariância:**

Você pode usar um “espaço de cães” onde um “espaço de animais” é esperado, porque todos os cães são animais. Mas você não pode usar um “espaço de animais” onde um “espaço de cães” é esperado, porque ele pode conter animais que não são cães.

- **Contravariância** (pense em termos de funções):

Se você tem algo que pode lidar com **qualquer animal**, você pode usá-lo onde algo que lida **apenas com cães** é esperado. Mas não o contrário.

Exemplo de covariância:

```
class Animal {
  name: string;
  constructor(name: string) {
    this.name = name;
  }
}

class Dog extends Animal {
  breed: string;
  constructor(name: string, breed: string) {
    super(name);
    this.breed = breed;
  }
}

let animals: Animal[] = [];
let dogs: Dog[] = [];

// Arrays are covariant in TypeScript (but not type-safe)
animals = dogs; // allowed
dogs = animals; // error
```

Exemplo de contravariância:

```

class Animal {
    name: string;
    constructor(name: string) {
        this.name = name;
    }
}

class Dog extends Animal {
    breed: string;
    constructor(name: string, breed: string) {
        super(name);
        this.breed = breed;
    }
}

type Feed<T> = (animal: T) => void;

let feedAnimal: Feed<Animal> = animal => {
    console.log(animal.name);
};

let feedDog: Feed<Dog> = dog => {
    console.log(dog.breed);
};

// Intended contravariance:
feedDog = feedAnimal; // safe

// This depends on compiler settings:
feedAnimal = feedDog; // error only with strictFunctionTypes

```

Anotações de Variância Opcionais para Parâmetros de Tipo

A partir do TypeScript 4.7.0, podemos usar as palavras-chave `out` e `in` para sermos específicos sobre a anotação de variância.

Para Covariante, use a palavra-chave `out`:

```

type AnimalCallback<out T> = () => T; // T é Covariante aqui

```

E para Contravariante, use a palavra-chave `in`:

```
type AnimalCallback<in T> = (value: T) => void; // T é
Contravariante aqui
```

Assinaturas de Índice de Padrão de String de Template (Template String Pattern Index Signatures)

Assinaturas de índice de padrão de string de template permitem definir assinaturas de índice flexíveis usando padrões de string de template. Este recurso nos permite criar objetos que podem ser indexados com padrões específicos de chaves de string, fornecendo mais controle e especificidade ao acessar e manipular propriedades.

O TypeScript a partir da versão 4.4 permite assinaturas de índice para símbolos e padrões de string de template.

```
const uniqueSymbol = Symbol('description');

type MyKeys = `key-${string}`;

type MyObject = {
  [uniqueSymbol]: string;
  [key: MyKeys]: number;
};

const obj: MyObject = {
  [uniqueSymbol]: 'Chave de símbolo único',
  'key-a': 123,
  'key-b': 456,
};

console.log(obj[uniqueSymbol]); // Chave de símbolo único
console.log(obj['key-a']); // 123
console.log(obj['key-b']); // 456
```

O Operador satisfies

O `satisfies` permite verificar se um determinado tipo satisfaz uma interface ou condição específica. Em outras palavras, ele garante que um tipo possui todas as propriedades e métodos exigidos de uma interface específica. É uma maneira de garantir que uma variável se encaixe na definição de um tipo. Aqui está um exemplo:

```
type Columns = 'name' | 'nickName' | 'attributes';
```

```
type User = Record<Columns, string | string[] | undefined>;
```

```
// Anotação de Tipo usando `User`
```

```
const user: User = {  
  name: 'Simone',  
  nickName: undefined,  
  attributes: ['dev', 'admin'],  
};
```

```
// Nas linhas seguintes, o TypeScript não conseguirá inferir corretamente
```

```
user.attributes?.map(console.log); // A propriedade 'map' não existe no tipo 'string | string[]'. A propriedade 'map' não existe no tipo 'string'.
```

```
user.nickName; // string | string[] | undefined
```

```
// Asserção de tipo usando `as`
```

```
const user2 = {  
  name: 'Simon',  
  nickName: undefined,  
  attributes: ['dev', 'admin'],  
} as User;
```

```
// Aqui também, o TypeScript não conseguirá inferir corretamente
```

```
user2.attributes?.map(console.log); // A propriedade 'map' não existe no tipo 'string | string[]'. A propriedade 'map' não existe no tipo 'string'.
```

```
user2.nickName; // string | string[] | undefined
```

```
// Usando operadores `satisfies` podemos inferir os tipos corretamente agora
```

```
const user3 = {
  name: 'Simon',
  nickName: undefined,
  attributes: ['dev', 'admin'],
} satisfies User;

user3.attributes?.map(console.log); // TypeScript infere
corretamente: string[]
user3.nickName; // TypeScript infere corretamente: undefined
```

Importações e Exportações Apenas de Tipo (Type-Only Imports and Export)

Importações e Exportações Apenas de Tipo permitem importar ou exportar tipos sem importar ou exportar os valores ou funções associados a esses tipos. Isso pode ser útil para reduzir o tamanho do seu bundle.

Para usar importações apenas de tipo, você pode usar a palavra-chave `import type`.

O TypeScript permite usar extensões de arquivo de declaração e implementação (.ts, .mts, .cts e .tsx) em importações apenas de tipo, independentemente das configurações de `allowImportingTsExtensions`.

Por exemplo:

```
import type { House } from './house.ts';
```

As seguintes são formas suportadas:

```
import type T from './mod';
import type { A, B } from './mod';
import type * as Types from './mod';
export type { T };
export type { T } from './mod';
```

declaração using e Gerenciamento Explícito de Recursos (Explicit Resource Management)

Uma declaração `using` é um vínculo imutável com escopo de bloco, semelhante ao `const`, usado para gerenciar recursos descartáveis (disposable). Quando inicializado com um valor, o método `Symbol.dispose` desse valor é registrado e subsequentemente executado ao sair do escopo de bloco envolvente.

Isso é baseado no recurso de Gerenciamento de Recursos do ECMAScript, que é útil para realizar tarefas essenciais de limpeza após a criação do objeto, como fechar conexões, excluir arquivos e liberar memória.

Notas:

- Devido à sua introdução recente na versão 5.2 do TypeScript, a maioria dos ambientes de execução carece de suporte nativo. Você precisará de polyfills para: `Symbol.dispose`, `Symbol.asyncDispose`, `DisposableStack`, `AsyncDisposableStack`, `SuppressedError`.
- Além disso, você precisará configurar seu `tsconfig.json` da seguinte forma:

```
{
  "compilerOptions": {
    "target": "es2022",
    "lib": ["es2022", "esnext.disposable", "dom"]
  }
}
```

Exemplo:

```
//@ts-ignore
Symbol.dispose ??= Symbol('Symbol.dispose'); // Polyfill simples
```

```

const doWork = (): Disposable => {
  return {
    [Symbol.dispose]: () => {
      console.log('disposto (disposed)');
    },
  };
};

console.log(1);

{
  using work = doWork(); // Recurso é declarado
  console.log(2);
} // Recurso é descartado (ex: `work[Symbol.dispose]()` é avaliado)

console.log(3);

```

O código registrará:

```

1
2
disposto (disposed)
3

```

Um recurso elegível para descarte deve aderir à interface Disposable:

```

// lib.esnext.disposable.d.ts
interface Disposable {
  [Symbol.dispose](): void;
}

```

As declarações `using` registram as operações de descarte de recursos em uma pilha, garantindo que sejam descartadas na ordem inversa da declaração:

```

{
  using j = getA(),
    y = getB();
  using k = getC();
} // descarta `C`, depois `B`, depois `A`.

```

Os recursos têm garantia de serem descartados, mesmo que ocorra código subsequente ou exceções. Isso pode levar o descarte a potencialmente lançar uma exceção, possivelmente suprimindo outra. Para manter informações sobre erros suprimidos, uma nova exceção nativa, `SuppressedError`, é introduzida.

declaração `await using`

Uma declaração `await using` lida com um recurso descartável de forma assíncrona. O valor deve ter um método `Symbol.asyncDispose`, que será aguardado ao final do bloco.

```
async function doWorkAsync() {
    await using work = doWorkAsync(); // Recurso é declarado
} // Recurso é descartado (ex: `await work[Symbol.asyncDispose]()`
é avaliado)
```

Para um recurso descartável de forma assíncrona, ele deve aderir à interface `Disposable` OU `AsyncDisposable`:

```
// lib.esnext.disposable.d.ts
interface AsyncDisposable {
    [Symbol.asyncDispose](): Promise<void>;
}

//@ts-ignore
Symbol.asyncDispose ??= Symbol('Symbol.asyncDispose'); // Polyfill
simples

class DatabaseConnection implements AsyncDisposable {
    // Um método que é chamado quando o objeto é descartado
    assincronamente
    [Symbol.asyncDispose]() {
        // Fecha a conexão e retorna uma promise
        return this.close();
    }

    async close() {
        console.log('Fechando a conexão...');
        await new Promise(resolve => setTimeout(resolve, 1000));
        console.log('Conexão fechada.');
```

```

    }
}

async function doWork() {
    // Cria uma nova conexão e descarte-a assincronamente quando
    // ela sair do escopo
    await using connection = new DatabaseConnection(); // Recurso é
    // declarado
    console.log('Fazendo algum trabalho...');
} // Recurso é descartado (ex: `await
    connection[Symbol.asyncDispose]()` é avaliado)

doWork();

```

O código registra:

```

Fazendo algum trabalho...
Fechando a conexão...
Conexão fechada.

```

As declarações `using` e `await using` são permitidas em Instruções: `for`, `for-in`, `for-of`, `for-await-of`, `switch`.

Atributos de Importação (Import Attributes)

Os Atributos de Importação do TypeScript 5.3 (rótulos para importações) dizem ao ambiente de execução como lidar com módulos (JSON, etc.). Isso melhora a segurança garantindo importações claras e se alinha com a Política de Segurança de Conteúdo (CSP) para carregamento de recursos mais seguro. O TypeScript garante que eles sejam válidos, mas deixa o ambiente de execução lidar com sua interpretação para manipulação específica de módulos.

Exemplo:

```
import config from './config.json' with { type: 'json' };
```

com importação dinâmica:

```
const config = import('./config.json', { with: { type: 'json' } });
```

Verificação de Sintaxe de Expressões Regulares

Desde o TypeScript 5.5.4, ele verifica literais de expressões regulares em busca de erros comuns em tempo de compilação (por exemplo, sintaxe inválida, referências invertidas, recursos não suportados pela sua versão de destino do JavaScript). Isso ajuda a detectar erros mais cedo, mas não verifica novas strings RegExp(...).

```
let r = /(a)\2/; // Erro: Esta referência invertida se refere a um grupo que não existe.
```

import defer

`import defer` permite carregar um módulo, mas adiar sua execução até que você realmente use algo dele. Isso ajuda a evitar trabalho desnecessário e efeitos colaterais.

- Funciona apenas com: `import defer * as name from "module"`
- O código é executado somente quando você acessa uma exportação

```
// arquivo: a.ts
console.log('executando!');
export const x = 1;
```

```
// arquivo: main.ts
// prettier-ignore
import defer * as a from "./a.js";
console.log('iniciando'); // nada de a.ts ainda
console.log(a.x); // agora imprime "executando!", depois 1
```